

SC4053 Blockchain Technology

Development Project

Decentralized Exchanges

SCEL, SC4053 Blockchain Technology, Nanyang Technological University

Version 1.0

Prepared by: Muhammadh Faheem Faizal (U2320336K), Kumar Advaith (U2323503B)

SC4053 DEX: AI Audit Report

Date: 8 November 2026. The report describes the submitted codebase: the trading engine, factories and deployment tooling in `contracts/`, and the Next.js front end with all three bonus features in `web/`.

Team:

- Person 1: trading engine (`OrderBook.sol`, its interface, specification and tests).
- Person 2: `TokenFactory`, `DemoToken`, `MarketFactory`, deployment script and exporter, Next.js front end, README.

Brief: the SC4053 Development Project brief (AY2026-2027 Semester 1, Option 1: DEX). This report follows the brief's eight required sections (section 2.1) and the four-part breakdown for major issues (section 2.2).

Sources. Each statement in this report is backed by a committed record and cited by file path or commit hash. Person 1's sections draw on `docs/ai-audit/prompt-log.md`, issues 001 to 007 and the engine evidence; Person 2's draw on `prompt-log-person2.md`, `tooling-and-methodology.md`, issues P2-001 to P2-017 and the `test-evidence/person2/` files. Automated tests, real-chain runs, manual MetaMask runs and read-only chain checks prove different things, so the text states which kind of evidence supports each claim (sections 6.5 to 6.10). The one exception is Person 1's use of the claude.ai chat (the executive summary, 1.6, 2.2, 2.4, Figure 1, 3.1, 3.2 and 8.3 item 7): that chat transcript is not committed, so those statements are Person 1's own account.

Executive summary

- Product: a minimal, non-custodial order-book DEX on a local Ethereum chain (Anvil). Anyone can issue an ERC20 (`TokenFactory`) and list a token pair (`MarketFactory`, one `OrderBook` per pair). Traders place limit orders from MetaMask through a Next.js app. Matching runs on-chain at the maker's price with price-time priority, partial fills and per-fill refunds of price improvement.
- All three bonuses are implemented in the engine, with no change to the interface frozen on 1 Nov:
  - cancellation of unfilled amounts (`cancelOrder`)
  - batched multi-party matching (`matchOrders`; 4 buyers and 4 sellers settled in one transaction)
  - conditional stop-limit orders (`placeStopLimitOrder`, `triggerStopOrders`)

- All three bonus features also have a front end (Person 2: cancel in P2-8B.2, batch matching in P2-8B.3, stop-limit orders in P2-8B.4). Each was verified with unit, real-chain and mock-wallet browser tests, and the owner ran the happy path of each in real MetaMask (sections 6.5, 6.7 and 6.9). The MetaMask negative cases (rejection, switching account during a request, an RPC outage, a chain reset) were not run. The owner's Bonus 3 run exposed one front-end defect, a stop cancelled before activation labelled "Activated"; it was fixed in P2-8C (issue P2-017, section 6.10). The regenerated ABI is in `web/` (`e375ff6`).
- AI use, and who decided: over 54 logged sessions (31 for Person 1, 23 for Person 2), every session was started and scoped by one of us. We used Claude Code as the coding assistant, steered by specification files read every session (`CLAUDE.md`, `SPEC.md`, the frozen `CONTRACT_INTERFACE.md`, per-phase prompts). Person 1 also used Claude in the claude.ai chat, minimally, as planner and reviewer (Section 2.4). We set the scope, approved every phase plan, and made every design trade-off: among them lot and tick sizes, one shared trigger budget, `bookSeq` priority, the escrow order, auto-cancel for issue 006, accepting issue 007, RainbowKit 2 and a separate `MarketFactory` (Section 2.6 describes the human effort and lists the decisions with their log entries). From Phase 3a on, a fresh, read-only AI reviewer checked each engine phase, and we decided what to do with each finding.
- Problems found in AI output: 24 issues are logged in the four-part format: 7 engine (001 to 007) and 17 Person 2 (P2-001 to P2-017).
    - Highest: 006 (High availability: one 102-wei stop blocked every stop trigger, fixed) and 004 (High test quality: three injected matching bugs would have passed the suite; the code itself was correct). For Person 2, two Medium deployment-tooling defects (P2-013, P2-014, fixed).
    - One Medium engine issue is accepted by design and pinned by a test: 007 (a limit order can be sandwiched through the fill cap; the loss is bounded by the victim's limit).
- Validation:
    - Tests: 335 Foundry tests (245 engine, 90 Person 2) and 604 front-end unit tests passed (76 more are gated real-chain tests that skip without a scratch chain; 576 and 75 after Bonus 3, and 410 and 48 before the bonus UI).
    - Invariants: a stateful invariant suite with an independent reference ledger held for 1,000 runs × depth 128 (128,000 calls, 0 reverts).
    - Coverage: `OrderBook.sol` 100% on lines, statements, branches and functions.
    - Static analysis: Slither found 0 real issues in the engine.
    - Mutation testing: each phase's mutants were caught. The full set of 67 hand-written mutants, run once at the end, caught 65; the 2 survivors are documented equivalents.

    - Reviews: a fresh-context AI reviewer checked each engine phase from Phase 3a, and a final independent AI audit of the engine found no Critical or High. Person 2's contracts, deploy tooling and first front-end phases had fresh-context AI reviews in P2-1 to P2-6 (P2-6.1 and P2-7 had none); the later front-end phases were checked by tests and the owner's manual runs rather than by a separate reviewer.
    - Real chain: scratch-chain `cast` journeys and a one-command demo replay passed.
- Not proven:
    - no human security audit, and the final audit covered the engine only
    - no public testnet
    - the MetaMask rejection, account-switch, outage and chain-reset cases were not run (`docs/TESTING.md`, Section F)
    - a mock wallet is not MetaMask, so the browser suites say nothing about MetaMask itself

Contents

1. Initial Specification and System Scope
2. Development Methodology and AI Tooling Infrastructure
3. Specification Engineering and Prompt Logs
4. AI-Generated Architecture
5. Problems Discovered and Vulnerability Analysis
6. Validation Process and Testing Strategy
7. Improvements and Refinements
8. Lessons Learned

- Requirements traceability
- Evidence index

List of figures

Figure	Title	Section
1	AI-assisted workflow per phase	2.4
2	Engine timeline from git and log dates	2.5
3	System architecture and trust boundary	4.1
4	Order lifecycle	4.2
5	A buy that partly fills, then rests	4.2
6	Order book and stop-list structure	4.2

Figure	Title	Section
7	`triggerStopOrders` flow	4.2
8	Multi-party batch match in one transaction	4.2
9	Engine issues 001 to 007 by phase and severity	5.1
10	Verification evidence map	6.1

Every figure is drawn from the committed code, tests, logs and git history. The Mermaid sources and the render script are in `diagrams/`.

---

1. Initial Specification and System Scope

1.1 Problem and chosen option

We chose **Option 1: Decentralized Exchange (DEX)** from the SC4053 development project brief. The brief's problem statement is that centralized exchanges hold users' private keys, which makes them honeypots for theft or misuse.

Our goal was a minimal viable DEX on Ethereum in which:

- users keep their keys in their own wallets (MetaMask)
- a smart contract acts only as a non-custodial marketplace that matches and settles trades between peers

We are a two-person team, and we split the work before writing any code:

Role	Owner	Scope
Trading engine	Person 1	`OrderBook.sol` : order placement, escrow, matching, settlement, partial fills, cancellation, batch matching, conditional orders, engine security and contract tests
Tokens, deployment and frontend	Person 2	ERC20 issuance (`DemoToken` , `TokenFactory`), market listing, deployment scripts and ABI export, the Next.js frontend, wallet integration, frontend tests and documentation
Shared	Both	The contract interface, integration, cross-review, the AI Audit Report and the demo

1.2 Required features (from the brief)

Brief requirement	What we intended to build
Asset issuer: issue ERC20 tokens	Demo tokens (NTU, SGDx) issued through a token factory
Submit buy and sell limit orders	`placeLimitOrder(side, price, amount)` for both sides

Brief requirement	What we intended to build
Automatic execution of matched orders, with partial fulfilment	Matching runs inside the placing transaction, and any remainder stays on the book
Bonus 1: cancel unfulfilled amounts	An owner-only `cancelOrder` that refunds only the unfilled escrow
Bonus 2: batched multi-party matching	`matchOrders(maxMatches)`, which settles fills between many traders in one bounded transaction
Bonus 3: conditional orders	Stop-limit orders that activate when the last trade price crosses a trigger

1.3 Intended smart contract logic

These were our design decisions before implementation:

- Where they were recorded: `SPEC.md` and the `CONTRACT_INTERFACE.md` draft, both committed on 28 Oct in `cd35f8b`.
- The freeze: the interface was frozen on 1 Nov (`63aee60`, Phase 2a), before any escrow, matching or bonus code. Phase 1's skeleton (`7c73490`, ten minutes earlier) was built against the draft.

Decision	Our intended rule	Reason
Markets	One order book contract per token pair (base and quote)	Keeps matching simple. More pairs means more contracts, not more routing logic
Escrow	Both sides fully funded at placement: sellers lock base tokens, buyers lock quote tokens at their limit price	An order should never be accepted just because a wallet held enough tokens at one moment
Execution price	The resting (maker) order's price	Deterministic and easy to explain (Figure 5)
Price improvement	A buyer filled below their limit is refunded the difference	No funds left stuck in the contract
Priority	Better price first; at equal price, the older order first	Answers the brief's question of whether older orders deserve priority: yes. "Older" became the time an order enters the book (`bookSeq`, decided 28 Oct as O1), not its ID (Figure 6)
Who triggers execution	The trader placing the order pays the gas for matching; anyone can call batch matching or stop triggering	Answers the brief's question on how, how often and by whom execution is triggered: a contract cannot run on its own (Figure 8)
Bounded work	Every loop over orders has a fixed upper limit	Predictable gas, and no transaction that can never complete
Conditional orders	Stop-limit, triggered by the DEX's own last trade price, with an explicit "no price yet" state (`hasTradePrice`)	Avoids an external oracle. An uninitialized zero must never look like a real price (Figure 7)

Decision	Our intended rule	Reason
Arithmetic	Integer base units only (18-decimal tokens), with rounding that always leaves the contract solvent	Solidity has no floating point
Fees and admin	No fees, no owner and no admin path to user funds	Simpler accounting, and consistent with non-custody (Figure 3)

The resulting lifecycle is shown in Figure 4.

1.4 Intended UI features

Person 2's front-end scope was fixed in Prompt 00, a read-only planning session of 5 Nov (three plan revisions, `prompt-log-person2.md`), before any front-end code, and written down in `PRODUCT.md` (first committed on 5 Nov in `3aba573`) and `DESIGN.md`. The intended product was:

- Wallet connection with clear handling of the wrong network, rejected transactions, and pending, confirmed or reverted transactions. The app reads the chain itself and the wallet only signs.
- Token issuance and market listing: create an ERC20 (18 decimals, fixed supply, no owner) through `TokenFactory`, and list a token pair through `MarketFactory`, which rejects a duplicate or reversed pair.
- Trading views: wallet balances shown separately from escrowed amounts, an order book and depth view read from the contract, buy and sell limit-order forms with the ERC20 approval as a separate step, and the user's own orders with their status and fills.
- Accurate status: the screens distinguish a wallet authorisation, a submitted transaction, a confirmed receipt, the decoded trade and the current order status, and show no figure the contracts cannot produce (no USD value, no 24-hour change).
- Controls for the bonus features: cancel, batch match and stop-limit orders, planned in `docs/BONUS_INTEGRATION.md` and built after the engine finished them (Section 7).

1.5 Explicitly out of scope

- Trading features:
  - automated market makers and liquidity pools
  - market orders and trading fees
  - external price oracles
  - cross-chain features
- Architecture: contract upgradeability, off-chain matching and a backend database.
- Deployment: no real-money or mainnet deployment. The system targets a local Anvil chain.

`SPEC.md` section 1 lists the engine's out-of-scope items.

1.6 How AI was involved at this stage

We set the core rules; AI helped write them down and test them. The specification was not written without AI, but the decisions in it were ours.

What we did first, ourselves: we read the brief and fixed the option, the role split and the core trading rules.

How AI was used before any contract code: to stress-test the plan and turn it into written specification files, in two steps.

- The plan: Claude in the claude.ai chat app (used minimally), before Claude Code was engaged.
  - What it did: Person 1 used one long chat, running from 28 Oct to 8 Nov, to stress-test the brief-based plan. The chat then rewrote it as the Person 1 engine plan, `docs/DEX_Trading_Engine_Plan.md`.
  - What came out of that review: buy orders are escrowed too, and the stop trigger has an explicit initial state (`hasTradePrice`). We decided both ourselves, and both are in the plan.
  - Evidence: this is the owner's account; the chat transcript is not in the repository. The repository agrees on the order of events: Claude Code's first session, on 28 Oct, opens by reading this plan, which was already in `docs/` but not yet committed (prompt log, Phase 0).
- The specification files. `CLAUDE.md`, `SPEC.md` and the `CONTRACT_INTERFACE.md` draft were written in Claude Code's Phase 0 plan-mode session on 28 Oct (`cd35f8b`). Claude Code raised seven specification gaps, (a) to (g). We decided each one: we kept its fix for six, replaced its fix for gap (b) with lot and tick sizes, and overruled one other proposal, the trigger budget (Section 3.2). We also decided the `bookSeq` tie-break (O1) the same day.
- The first contract code. Claude Code was asked to write it only on 1 Nov (Phase 1), after those files were committed. It therefore built against our specification rather than inventing one.

Person 2's side. The records show no written specification by Person 2 before AI was engaged. Person 2's scope was set in Prompt 00 (5 Nov, Opus 5.5, read-only), where the AI produced three plan revisions and the owner made the decisions recorded in the log: RainbowKit 2 with wagmi 2, and `MarketFactory` as a separate Person 2 contract that leaves `OrderBook.sol` and the frozen interface untouched. `PRODUCT.md` and `DESIGN.md` followed, written with Claude Code.

What predates AI use: only the repository's initial commit (`095b2af`, 24 Oct, a one-line README).

---

2. Development Methodology and AI Tooling Infrastructure

2.1 Team and division of labour

Area	Owner	Files
Trading engine	Person 1	`contracts/src/OrderBook.sol`, `contracts/src/interfaces/`, `contracts/test/OrderBook*`, `contracts/test/invariant/`, `contracts/test/mocks/`, `SPEC.md`, `docs/ENGINE_PHASES.md`
Factories, deployment, front end, docs	Person 2	`TokenFactory`, `DemoToken`, `MarketFactory`, `contracts/script/`, `web/`, `README.md`
Shared (both must agree)	Both	`CLAUDE.md`, `CONTRACT_INTERFACE.md`, `contracts/foundry.toml`

- The interface was the contract between the two halves. It was frozen on 1 Nov (Person 2's approval was verbal, recorded in Person 1's log) and has not changed in signature since.
- Cross-boundary edits were few and named. Most needed changes were written down instead: in Person 2's audit (`p2-8a-audit.md`) and in Person 1's handoff (`docs/ENGINE_HANDOFF.md`).
- Person 1's sessions did edit three things in Person 2's area:
  - `web/lib/contracts/abis.ts`, regenerated with Person 2's own exporter (`e375ff6`). Person 1 authorised this; Person 2's approval is not recorded in the repository.
  - the engine section of `README.md` (`9c67419`, `5ca1b4a`)
  - a new `contracts/script/demo-journey.sh` (`9c67419`)

2.2 Tools

Tool	Person 1 (engine)	Person 2 (tokens, deployment, front end)
Claude (claude.ai chat)	Planner and reviewer, used minimally: one long chat, 28 Oct to 8 Nov (Section 2.4). Owner's account; the transcript is not in the repository	None recorded in Person 2's logs
Claude Code	Opus 5.5 (log header); plan mode for Phase 0. Coding assistant; also ran the independent subagent reviews	Opus 5.5, Sonnet 5.5 and Haiku 5.5 by phase (models per phase: `tooling-and-methodology.md`; some not recorded)

Tool	Person 1 (engine)	Person 2 (tokens, deployment, front end)
Independent reviewer	A fresh, read-only Claude Code subagent per phase from 3a to 7 (Phases 1 to 2b had none), plus a final audit	Fresh-context subagents in P2-1 to P2-6 (none for P2-5 wrap-up, P2-6.1, P2-7, the documentation sessions, P2-8A, P2-8A.1 and P2-8A.2)
Foundry (`forge`, `anvil`, `cast`): 1.8.6 in Person 1's log, 1.9.0 in Person 2's evidence	Unit, fuzz and invariant tests, gas, sizes, coverage, scratch-chain `cast` journeys	Contract tests, deploy script, scratch chains, `cast` journeys
Static analysis	Slither 0.11.6 on `OrderBook.sol` (Phase 8); `forge lint` triaged in the wrap-up	Slither 0.11.6 on Person 2's contracts and script; `forge lint`
Libraries	OpenZeppelin 5.7.0 (`SafeERC20`, `ReentrancyGuard`, `Math.mulDiv`); forge-std 1.17.0	Next.js, RainbowKit, wagmi, viem, TanStack Query; Vitest; `playwright-core` with a mock wallet
Skills and MCPs	Project commands `/review-engine` and `/log-issue`	Context7 (P2-3 to P2-5); shadcn and Awwwards MCPs; `frontend-design`, `ui-ux-pro-max`, `impeccable`, `taste-skill` (P2-4); `solidity-security` (P2-3, P2-8A); `react-best-practices` (P2-8A). The Playwright MCP was attempted but did not launch, and SkillSpector was not installed
Node.js	26.13.0, installed 7 Nov to run Person 2's tests and exporter against engine changes	Node 22.18 or newer

2.3 Custom AI infrastructure

- Specification files, read every session:
  - `CLAUDE.md` (source of truth, ownership, engine rules, "never weaken a failing test", "never claim something was run unless you ran it", log every session)
  - `SPEC.md` (accounting rules A1 to A8, lifecycle, per-function behaviour, known limitations)
  - the frozen `CONTRACT_INTERFACE.md`
  - `docs/ENGINE_PHASES.md` (build order, plus the verbatim prompts for Phases 1 to 4; later prompts are summarised in the prompt log)
- Project commands (`.claude/commands/`): `review-engine` (spawns an independent reviewer), `log-issue` (the four-part issue format, "use only output actually produced in this session") and `explain-diff`.
- Engine test infrastructure aimed at AI mistakes:
  - `OrderBookHarness`, to see internal links without changing the interface
  - `assertSolvent` and `assertBookSorted`, called after almost every test

    - hostile tokens: `ReentrantERC20` (re-enters on the Nth transfer, optionally swallowing failures) and `ViewSpyERC20` (records views mid-transfer)
    - a mutation script with 67 hand-written mutants: `docs/ai-audit/test-evidence/mutants.sh`
    - an invariant handler with an independent reference model (Section 6)
- Person 2's infrastructure:
    - a deploy script with a chain-id guard that asks the node
    - a dependency-free exporter (`contracts/script/export-web.mjs`) that refuses incomplete, stale or mismatched deployments
    - `scripts/check-local.sh`, and the scratch-chain helpers `scripts/seed-crossed-book.sh` and `scripts/seed-stop-orders.sh`
    - a mock-wallet browser harness (`web/e2e/mock-wallet.mjs`) and the suites built on it, plus real-chain integration suites that run the app's own transaction code against an unlocked scratch Anvil
    - the front end's prompt log, per-phase evidence files and one issue file per real defect (`docs/ai-audit/`)

2.4 Workflow per phase

Person 1 used a two-tier workflow. Below, "my" means Person 1. The chat tier was used minimally.

Tier	Tool	Role
Decision-maker	Person 1	Set the scope and schedule, approved each phase prompt and plan, decided every design trade-off and what to do with each review finding, and approved each merge
Planner and reviewer (used minimally)	Claude in the claude.ai chat (one chat, 28 Oct to 8 Nov)	Helped draft the phase prompts, reviewed Claude Code's plans and summaries, and suggested corrections
Coding assistant	Claude Code	Proposed plans, implemented each approved phase prompt, ran the tools, and ran the independent subagent reviews (`/review-engine`)

Evidence: the chat's role is the owner's account, and its transcript is not in the repository. The prompt log records each of the corrections below as a human decision or a human change to the AI's proposal. This report therefore attributes them as **my corrections (drafted with the chat assistant):**

Correction	Prompt-log entry
Lot and tick sizes instead of the AI's `AmountTooSmall`	Phase 0 planning, 28 Oct: "Human changes to the AI's proposal" (1)

Correction	Prompt-log entry
One shared fill budget for `triggerStopOrders`, instead of `maxCount × MAX_MATCHES_PER_TX`	Phase 0 planning, 28 Oct (2)
The `bookSeq` tie-break decided at once (O1), on the day it was found, before any code. SPEC.md had left it "to decide before Phase 6"	Phase 0 scaffold, 28 Oct: "Human decision, O1"
Escrow order: follow SPEC.md, pulling escrow before matching, where the Phase 2b prompt said to pull last	Phase 2b, 1 Nov: "Human decisions"
Issue 006: auto-cancel and refund, the review's option (a) of the three recorded	Phase 6, 7 Nov
Issue 007: document and pin by a test rather than change the engine	Phase 8, 8 Nov: "Human decision on M-1"

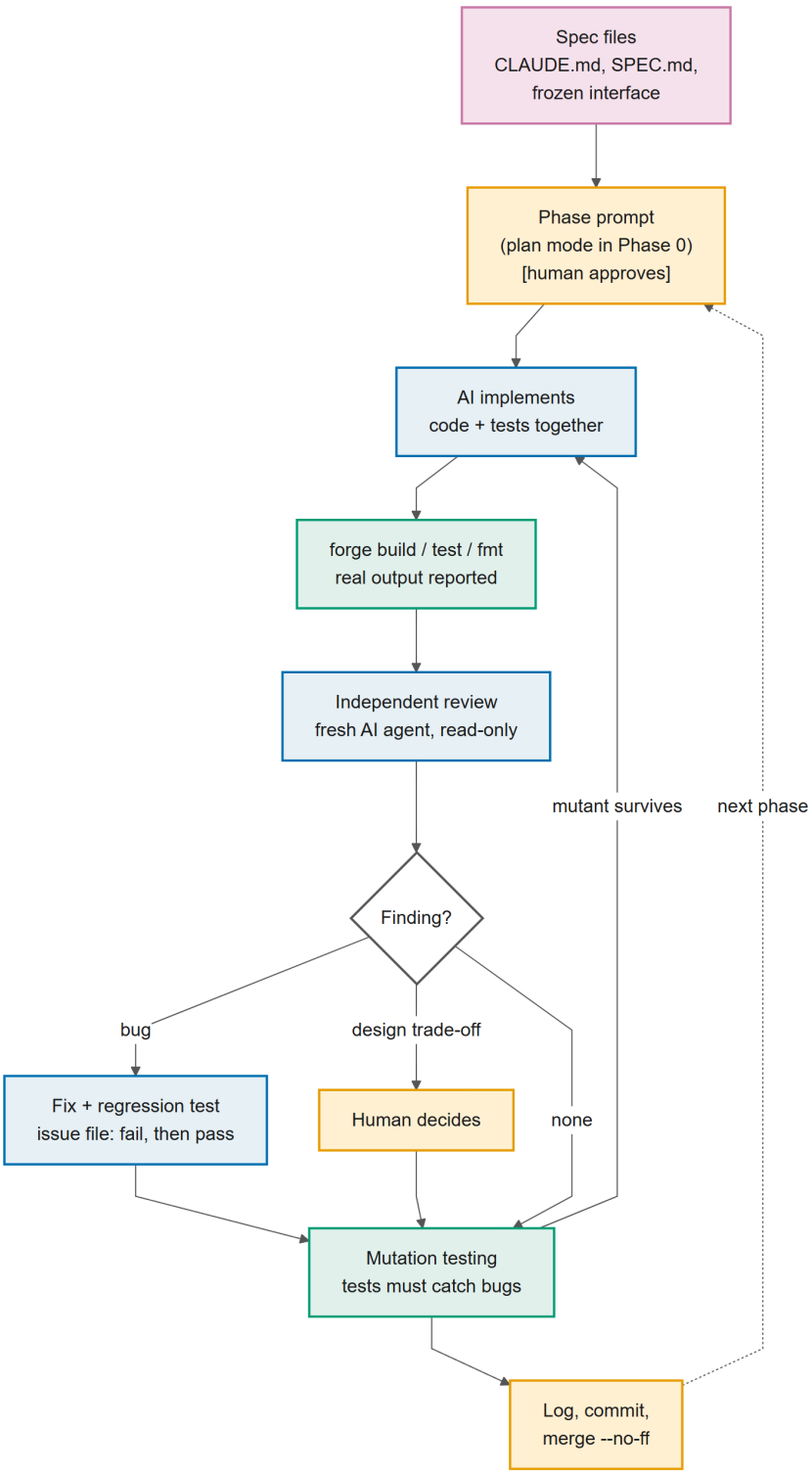


Figure 1: AI-assisted workflow per phase (engine). The human approves each phase prompt (`docs/ENGINE_PHASES.md`). Every AI change then passes real test output, a fresh read-only reviewer (from Phase 3a) and mutation testing before it is logged and merged. A surviving mutant sends the work back to implementation.

Person 2's records show the same pattern, without the chat tier (`prompt-log-person2.md`, `tooling-and-methodology.md`):

1. a planning session approved by the owner (Prompt 00 was read-only; later phases were planned first, and plans were revised when the owner added refinements, as in P2-7, P2-8B.4 and P2-8C)
2. implementation, with tests written alongside or ahead of the code
3. mutation checks and real-chain runs
4. for P2-1 to P2-6, a fresh-context AI review; later phases relied on tests and the owner's manual runs
5. the session, including the AI's mistakes, logged in `prompt-log-person2.md`

The owner chose the model per phase and decided which findings to fix and which to document.

2.5 Timeline

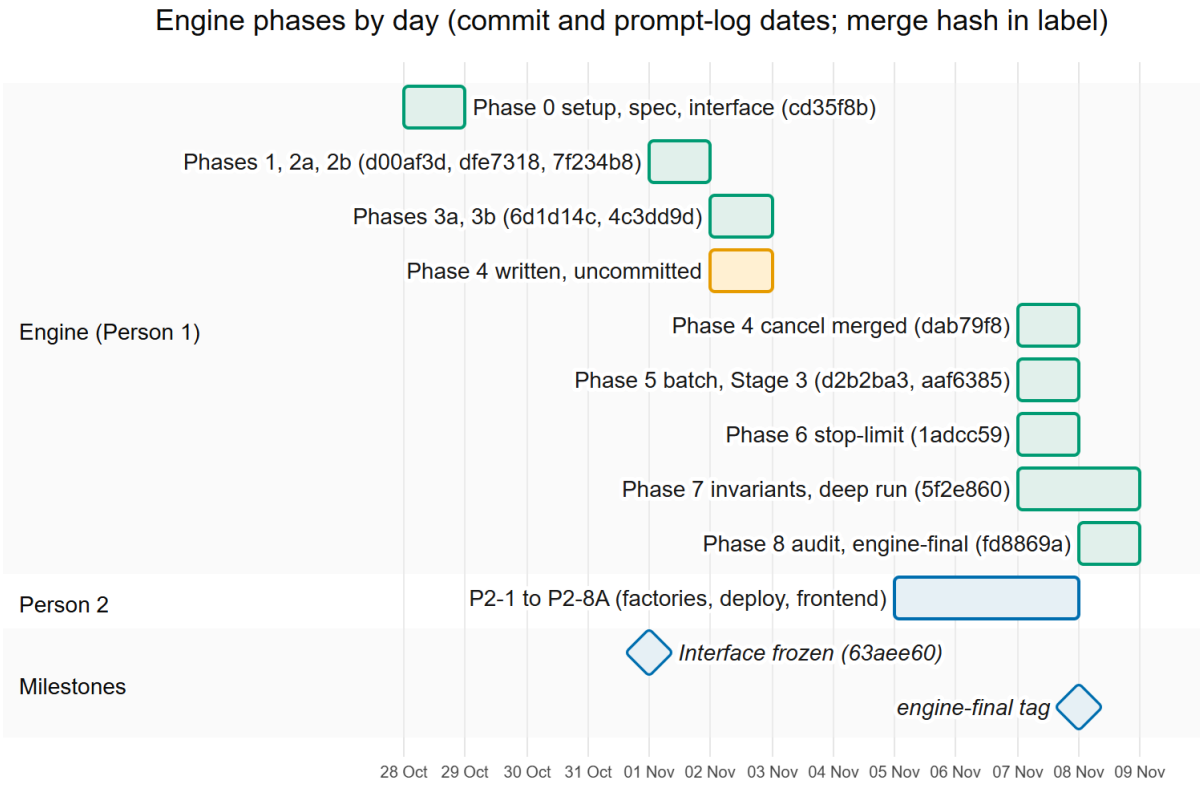


Figure 2: Engine timeline from git and prompt-log dates. Each bar is a calendar day with that phase's merge hash. Commit times are not working hours, so the bars show *when* the work landed, not how long it took. Phase 4 was written on 2 Nov but committed only after the 7 Nov recovery. The figure draws Person 2's bar from 5 Nov for three days. Person 2's non-merge commits actually run from 5 to 8 Nov (the bonus interfaces and P2-8C landed on 7 and 8 Nov, after the figure's cut-off; the initial README commit on 24 Oct is excluded), and their log has the detail.

2.6 Human effort and judgement

The direction, the constraints and the checking came from us, over two weeks of sessions, and the repository records it:

- Planning before any AI coding. Person 1's engine plan, `docs/DEX_Trading_Engine_Plan.md` (about 5,000 words), existed before Claude Code's first session on 28 Oct (Section 1.6). It fixed the scope, the interface, the accounting rules, the project rules that became `CLAUDE.md` (its section 9), a build order of nine phases each with a prompt and acceptance criteria (its section 11), the schedule and the cut order. Person 2 fixed the front-end scope in a read-only planning session before any front-end code (Prompt 00, 5 Nov), decided its open questions, and wrote `PRODUCT.md` and `DESIGN.md` from it.
- Every session was started and scoped by one of us. Person 1's prompt log records 31 sessions from 28 Oct to 8 Nov; Person 2's records 23 sessions from 5 to 8 Nov. The prompts set what to read first, which files the AI could and could not touch, what not to change without asking (any function, event or error of the frozen interface), and how to prove a result: real test output, regression tests shown failing before the fix and passing after, mutants, scratch chains only. Each engine phase prompt was approved by Person 1 before it ran (`docs/ENGINE_PHASES.md`).
- Decisions are recorded as ours. Person 1's log has 35 decisions under 13 "Human decisions" headings, and the later entries record what the human did. Each of Person 2's 23 entries records what the owner did: the approvals, the refinements added to the AI's plans before implementation, and the scope set for the session. The table below lists the decisions that changed the design.
- We checked the AI's work instead of trusting it. Every merge to `main` was a human decision. We asked for raw tool output rather than summaries (Section 8.2), corrected the AI where it was wrong (Section 3.2 and the "What the AI got wrong" line of every log entry), ran the real MetaMask sessions ourselves (Sections 6.5 to 6.9), and had this report re-verified against the repository claim by claim before submission (`report-verification.md`).

Decisions that changed the outcome, each recorded as a human decision in the prompt logs:

Decision	Who, when	What the AI had proposed or left open	Log entry
Lot and tick sizes, so every fill cost is exact	Person 1, 28 Oct	`AmountTooSmall`, which guarded only the total cost	`prompt-log.md`, Phase 0 planning

Decision	Who, when	What the AI had proposed or left open	Log entry
One shared fill budget for `triggerStopOrders`	Person 1, 28 Oct	`maxCount × MAX_MATCHES_PER_TX`	Phase 0 planning
Priority by `bookSeq`, decided before any code (O1)	Person 1, 28 Oct	SPEC left it open until Phase 6	Phase 0 scaffold, "Human decision, O1"
Phase 7 (verification) before Phase 6 (stop-limit), with a go/no-go point for the stop-limit bonus	Person 1, 1 Nov	n/a	Status check, 1 Nov
Guard `_unlink` in code, not only by a comment (issue 001)	Person 1, 1 Nov	a comment only	Phase 2a
Escrow pulled before matching, following SPEC	Person 1, 1 Nov	the Phase 2b prompt said pull last	Phase 2b
Keep a crossed book after a capped placement (F2), and document dust griefing (F3)	Person 1, 2 Nov	review findings with several options	Phase 3a
Keep the caps at 10 fills and 100 insert steps	Person 1, 2 Nov	n/a	Phase 3b
Each phase runs only its own mutants plus a smoke set; the full set runs once	Person 1, 7 Nov	n/a	Phase 5
Auto-cancel and refund for issue 006	Person 1, 7 Nov	three options from the review	Phase 6
Document and pin issue 007, no engine change	Person 1, 8 Nov	the audit's M-1	Phase 8
RainbowKit 2 with wagmi 2	Person 2, 5 Nov	drop RainbowKit for wagmi 3	`prompt-log-person2.md`, Prompt 00
`MarketFactory` as a separate contract, leaving the engine and interface untouched	Person 2, 5 Nov	n/a	Prompt 00
Plan refinements before implementation: P2-7 (two), P2-8A.2 ("do not treat invalidation as proof"), P2-8B.4 (two), P2-8C (the first plan sent back, three added)	Person 2, 6 to 8 Nov	the AI's first plans	P2-7, P2-8A.2, P2-8B.4, P2-8C
Verification only before any bonus UI, on scratch ports, protecting the main chain	Person 2, 8 Nov	n/a	P2-8B.0 and P2-8B.1
The real MetaMask runs, which found the stale-nonce problem and issue P2-017	Person 2, 6 to 8 Nov	n/a	P2-5 wrap-up, P2-8B.2 follow-up, P2-8B.4 follow-up

---

3. Specification Engineering and Prompt Logs

3.1 Specification files

File	Role
`CLAUDE.md`	Standing rules loaded every session
`SPEC.md`	The behaviour contract. Prompts cite its sections, and code and tests cite its rules (A1 to A8) by name
`CONTRACT_INTERFACE.md`	The frozen ABI. "Do not add any error, event or function that is not in the interface" forced fixes to stay inside the agreed surface (issues 001 and 006)
`docs/ENGINE_PHASES.md`	The phase table and the verbatim prompts for Phases 1 to 4 (later prompts are summarised in the prompt log). Tests that reviews found missing were added to the rows of later phases
`docs/MARKET_FACTORY.md`, `PRODUCT.md`, `DESIGN.md`	Person 2's specifications: the factory design, the product requirements and the visual system

3.2 Representative prompt chains

Engine (Person 1; full text in `docs/ai-audit/prompt-log.md`):

1. **Phase 0 planning, 28 Oct (plan mode):**
   - Claude Code raised seven specification gaps, (a) to (g). I (Person 1) reviewed each, kept most of its proposals and **replaced two**: the fix for gap (b), and a separate trigger-budget proposal (my corrections, drafted with the chat assistant):
     - `AmountTooSmall` became lot and tick sizes, because the AI's fix only guarded the total cost, not each partial fill.
     - A per-activation trigger budget became one shared budget, because the AI's version multiplied the worst-case gas.
   - Writing SPEC exposed the order-ID tie problem. SPEC left it open until Phase 6; I decided it the same day, which gave `bookSeq` (O1).
2. **Phase 3a, 2 Nov:** implementation, then a review with 11 findings. The phase prompt said "Log real findings with /log-issue, fix them with regression tests". Real defects became issues 002, 003 and 004. I decided F2 and F3: keep a crossed book after a capped placement (cleared by `matchOrders`), and document dust griefing rather than change `LOT_SIZE`. F5 and F6 were documented as limitations.
3. **Phase 6, 7 Nov:** a review found a 102-wei denial of service (H1). I chose the review's option (a), auto-cancel, over the other two recorded (document only, or a third list for blocked stops). Claude Code then reproduced the bug on the old line (3 FAIL), fixed it (3 PASS), added mutants P6x and P6y, and logged issue 006.

4. **Phase 7, 7 to 8 Nov:** a review of the AI's own tests found tautological ghost variables and a regression test that could not fail. The AI rebuilt the handler around an independent reference ledger. Mutants P7c to P7e prove the ledger catches wrong-recipient payouts; P7f to P7h prove the new matching-rule and auto-cancel checks.
5. **Phase 8, 8 Nov:** the final audit found M-1. I chose to document it rather than change the engine, and Claude Code reproduced it in the repo to the wei (issue 007).

Person 2 (full text in `prompt-log-person2.md`):

Chain	Iterations	Outcome
Prompt 00, planning	three plan revisions in read-only mode	scope, ownership, the MarketFactory proposal
P2-3 deployment	plan, implementation, mutants, review, fixes	a design contradicted by the Foundry docs was changed before coding; two Medium review findings fixed (P2-013, P2-014)
P2-5 limit orders	plan, implementation, fresh-context review	13 findings; P2-006 to P2-009
P2-7 hardening	plan, two owner refinements, implementation	evidence-based chain-reset detection; P2-012
P2-8A audit	plan, one owner refinement round	the coverage matrix and the first report draft
P2-8B.2 cancel	plan, implementation, mock-wallet and real-chain checks, then the owner's MetaMask run	the cancel flow built from the shared transaction machinery; the owner's real run found the wallet's stale nonce after an Anvil restart (an environment problem, not a code defect), which led to the troubleshooting documentation in Section 7
P2-8B.3 batch matching	plan, implementation, test-first runner, real-chain and browser suites	decisions fixed in the plan: send the market's own limit (not a user-chosen count), review and simulate before the wallet, never loop or run a keeper, show results only from the receipt's `Trade` events; one real defect (P2-016)
P2-8B.4 stop-limit	plan (re-planned once after the owner's two refinements), implementation, test-first helpers and runners, real-chain and browser suites, a separate scratch pair	decisions fixed in the plan: read the engine's real rules (eligibility only at the stop-list heads, equality triggers, auto-cancel at activation) rather than assume exchange conventions; keep *waiting for price* and *eligible, awaiting a Trigger stops transaction* distinct everywhere; escrow strictly from the LIMIT price; detect market-wide eligibility with the contract's own `triggerStopOrders` simulation because the engine has no view of its stop lists; never report activation as execution; fresh MetaMask accounts for the manual test. No audit issue arose while it was built; the owner's later MetaMask run exposed P2-017
P2-8C polish	plan (the first version was sent back so that the owner's refinements could be added), test-first stop-history classifier, new browser suite, full regression on a fresh scratch pair	decisions fixed in the plan: read a stop's history from `StopOrderTriggered` and `OrderCancelled` rather than infer it from its status, keep the history query bounded and tested under RPC failure, keep the label "Resting", one Title Case convention enforced by a test; issue P2-017

3.3 Context management

- The prompt logs carried state between sessions. Phase prompts start by reading `CLAUDE.md` and `SPEC.md`, and the 1 Nov resume also read the log.
- A resumed session started with a recovery check, not code. Person 1's 7 Nov session began with a recovery and sync: a WIP commit of the uncommitted work, Person 2's 24 new commits merged in, and a diff of the frozen files before any new code.
- Exact context-window contents are not preserved and are not claimed.

---

4. AI-Generated Architecture

4.1 System

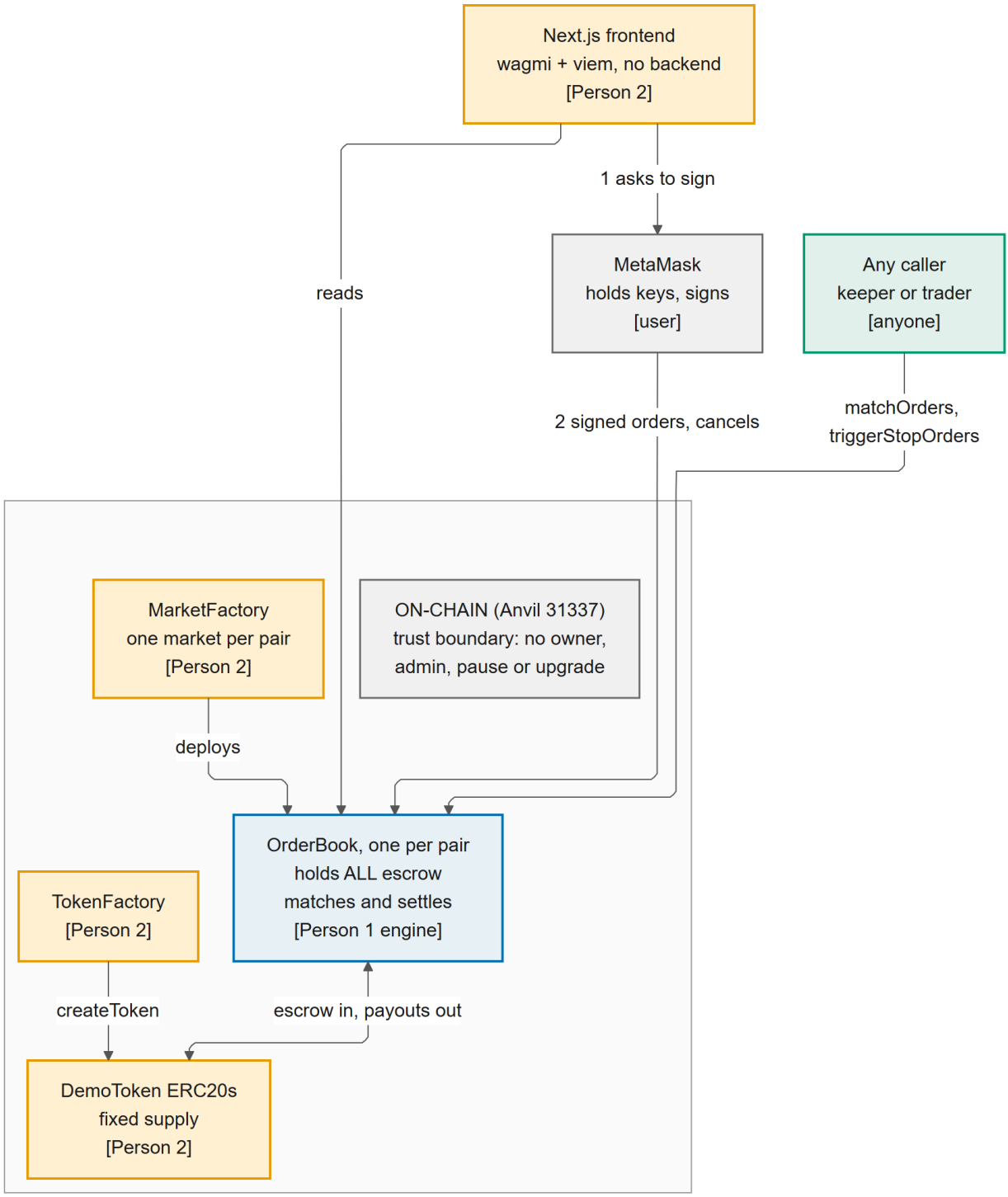


Figure 3: System architecture and trust boundary. Everything inside the box is on-chain with no owner or admin. The front end only reads and asks MetaMask to sign, and any caller can drive `matchOrders` and `triggerStopOrders`. The labels in brackets show who owns each part.

- Deployment: `Deploy.s.sol` (local only) deploys the factories and lists NTU/SGDx. The front end has no backend.
- Source: Person 2's README diagram, with the keeper path for the bonuses added.

Front-end integration of Bonus 2 (P2-8B.3). The Match Orders action is a thin addition to the transaction machinery the order ticket and Cancel already use: `lib/crossed-book.ts` decides from the market's own `getBestBid` and `getBestAsk` whether the live book is crossed; `lib/match-runner.ts` re-reads, simulates and sends one `matchOrders(MAX_MATCHES_PER_TX)` call through the shared `useTxFlow` reducer and `waitBounded`; `decodeMatchReceipt` reads the market's `Trade` logs; the result is reconciled with `getOrder` before and after the block. There is no new transaction framework, no backend and no keeper.

Front-end integration of Bonus 3 (P2-8B.4). Stop-limit orders reuse the same machinery rather than add a framework: `lib/stop-orders.ts` holds the engine's rules (buy stop eligible at last trade >= trigger, sell stop at last trade <= trigger, equality included, only after a first trade), the validation (escrow from the limit price) and the wording; `lib/stop-runner.ts` places the order through the shared approval step and lifecycle and verifies it from `StopOrderPlaced` and `getOrder` (pending, nothing filled, exact escrow); `lib/trigger-runner.ts` sends one `triggerStopOrders(10)` call and verifies each activated stop from `StopOrderTriggered`, `Trade`, `OrderCancelled` and `getOrder` before and after the block, telling the engine's own auto-cancel from a fill. The ticket's flow ends in a union outcome (`TicketOutcome`); cancelling a pending stop is the unchanged Bonus 1 runner and a crossed remainder is cleared by the unchanged Bonus 2 panel. There is no new transaction framework, no backend and no keeper.

4.2 Engine internals

- Call paths:
  - `placeLimitOrder` and `placeStopLimitOrder` share `_validate`, `_record` and `_pullEscrow`.
  - `placeLimitOrder` and `triggerStopOrders` match through `_matchTaker` (with a fill budget).
  - `_matchTaker` and `matchOrders` both settle through `_settleFill`, the only settlement path (maker price, A4 and the A5 refund).
  - `cancelOrder`, and an activated stop that cannot enter the book, both close through `_closeAndRefund`.
- State: `_orders`; the book heads with shared next and prev links; `_stopHead`; `_bookSeq`; `lastTradePrice`; `hasTradePrice`.
- An order is in at most one list, and its status says which: PendingTrigger orders are in a stop list, Open and PartiallyFilled orders in the book.
- Size: OrderBook 8,461 B runtime and MarketFactory 10,508 B, against the 24,576 B limit (`phase8/sizes.txt`).

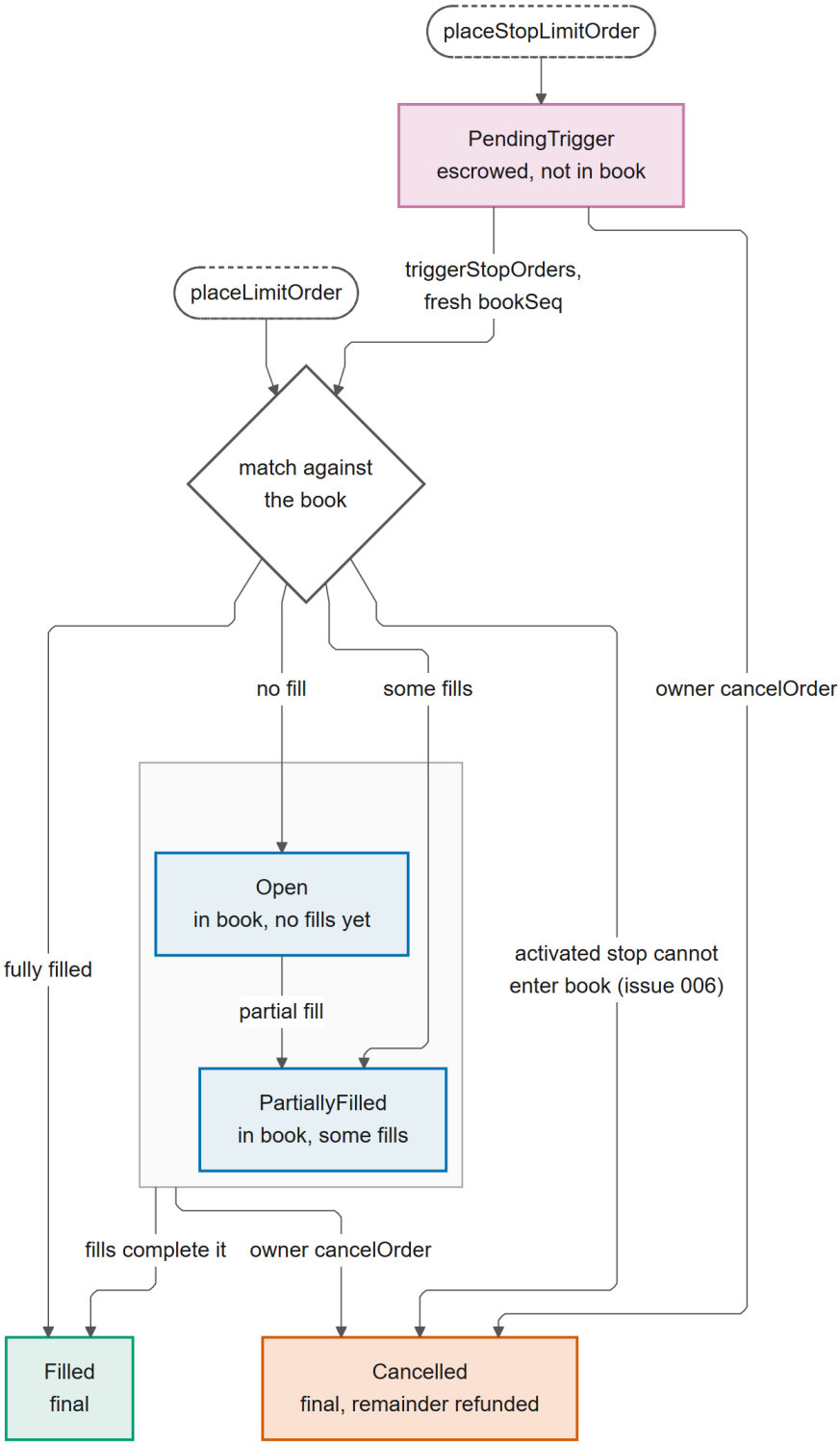


Figure 4: Order lifecycle. A limit order is matched at once. A stop-limit order waits as PendingTrigger until `triggerStopOrders` activates it with a fresh `bookSeq`. Filled and Cancelled are final, and a cancel always refunds the remainder, including the issue 006 auto-cancel.

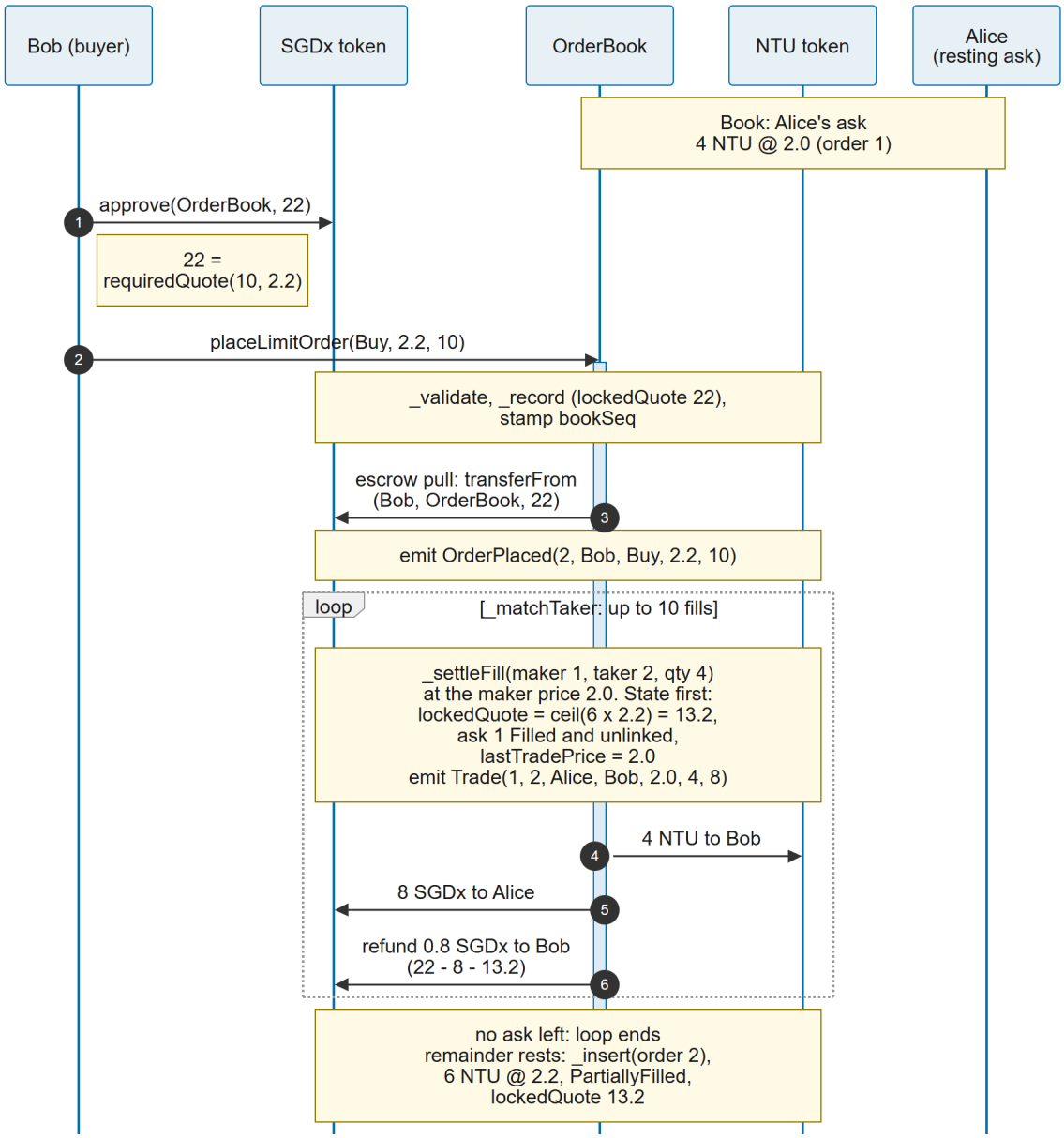


Figure 5: A buy that partly fills, then rests (rule A5). Bob escrows 22 SGDx for 10 NTU at 2.2 and fills 4 against Alice's ask at her price of 2.0. Settlement re-locks ceil(6 × 2.2) = 13.2 and refunds 0.8 before the 6 NTU remainder rests in the book.

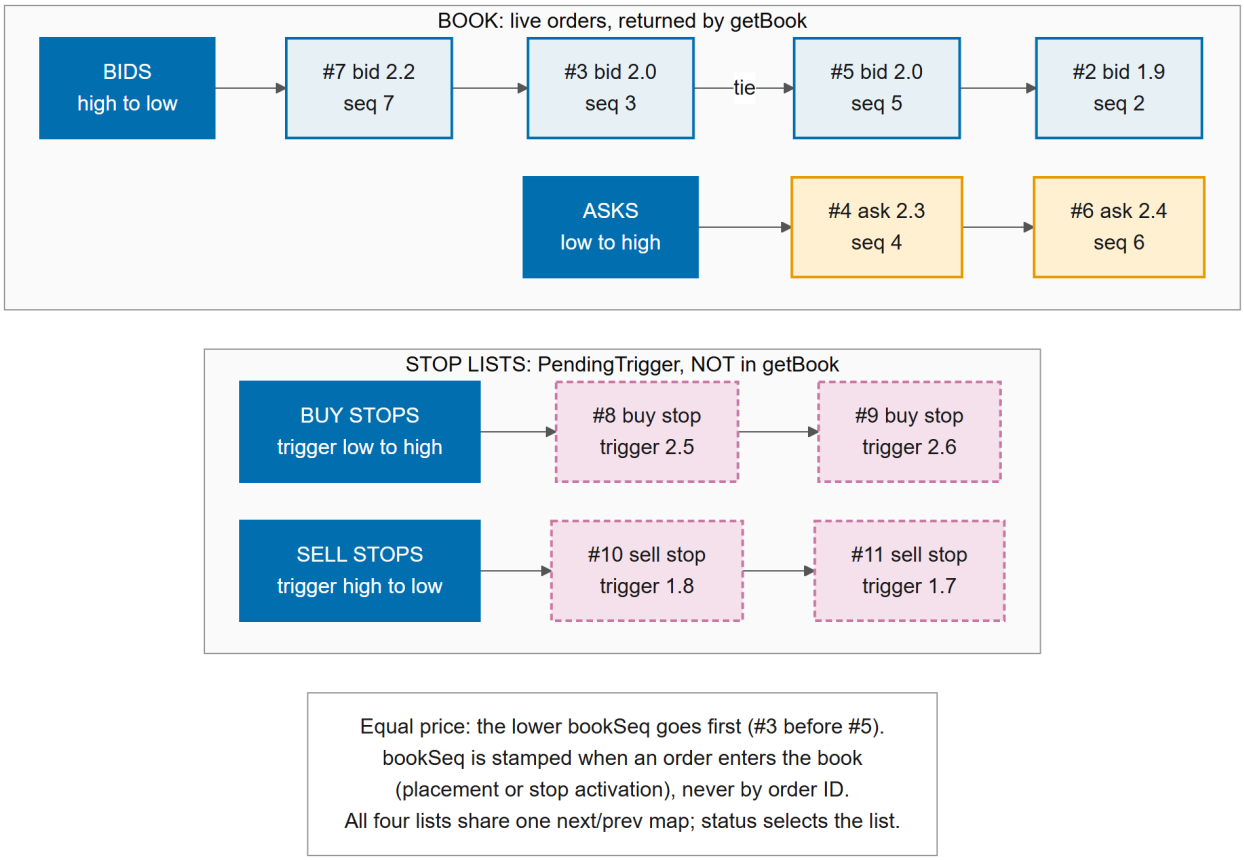


Figure 6: Order book and stop-list structure. There are four sorted linked lists over one shared next/prev map. Price ties go to the lower `bookSeq` (orders #3 and #5 at 2.0). Stop orders are held apart and are not returned by `getBook` until they activate.

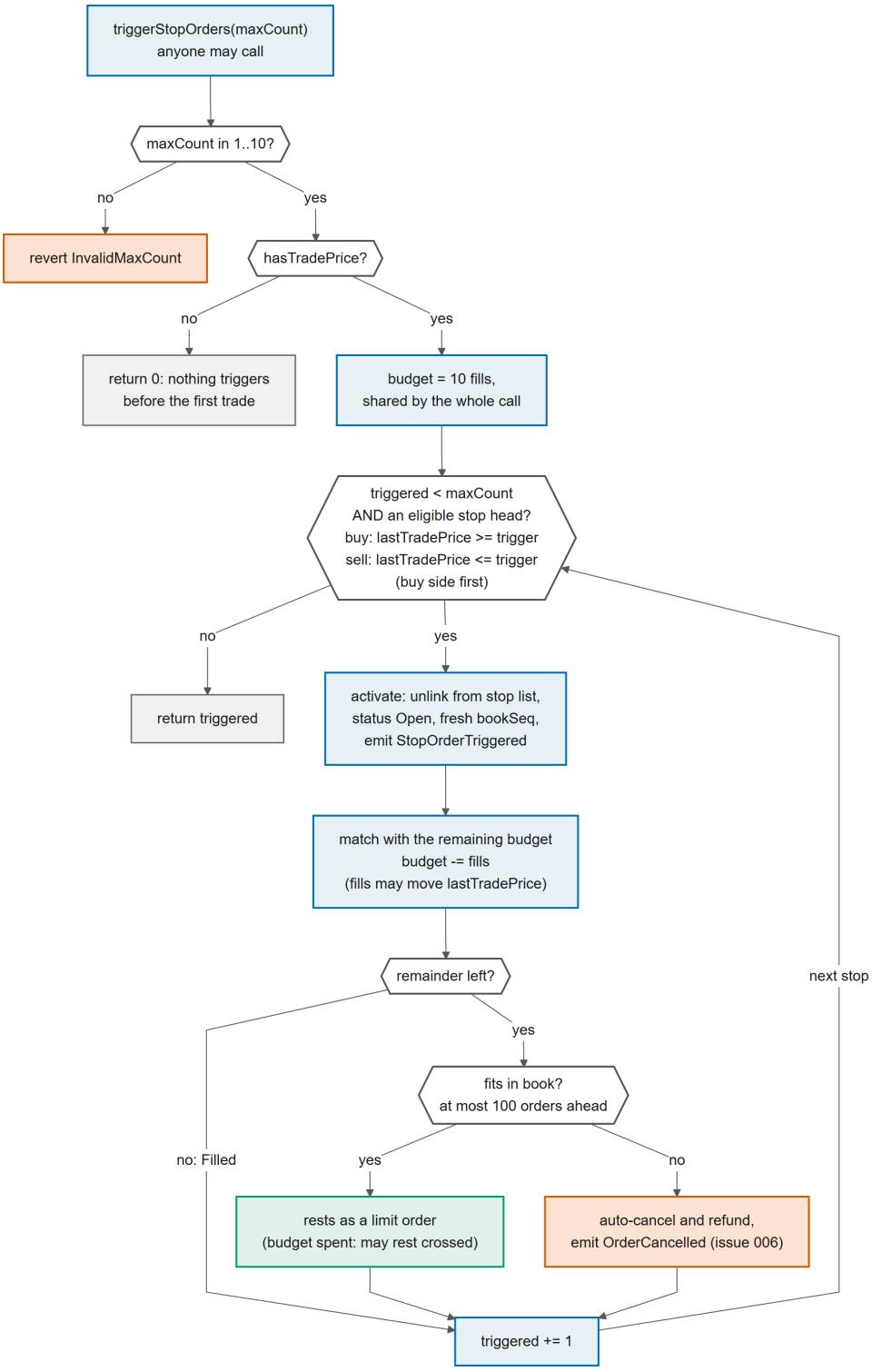


Figure 7: `triggerStopOrders` flow. One budget of 10 fills is shared by the whole call. The buy side is checked first. An activated remainder that cannot be placed within 100 insert steps is cancelled and refunded instead of reverting the call (the issue 006 fix).

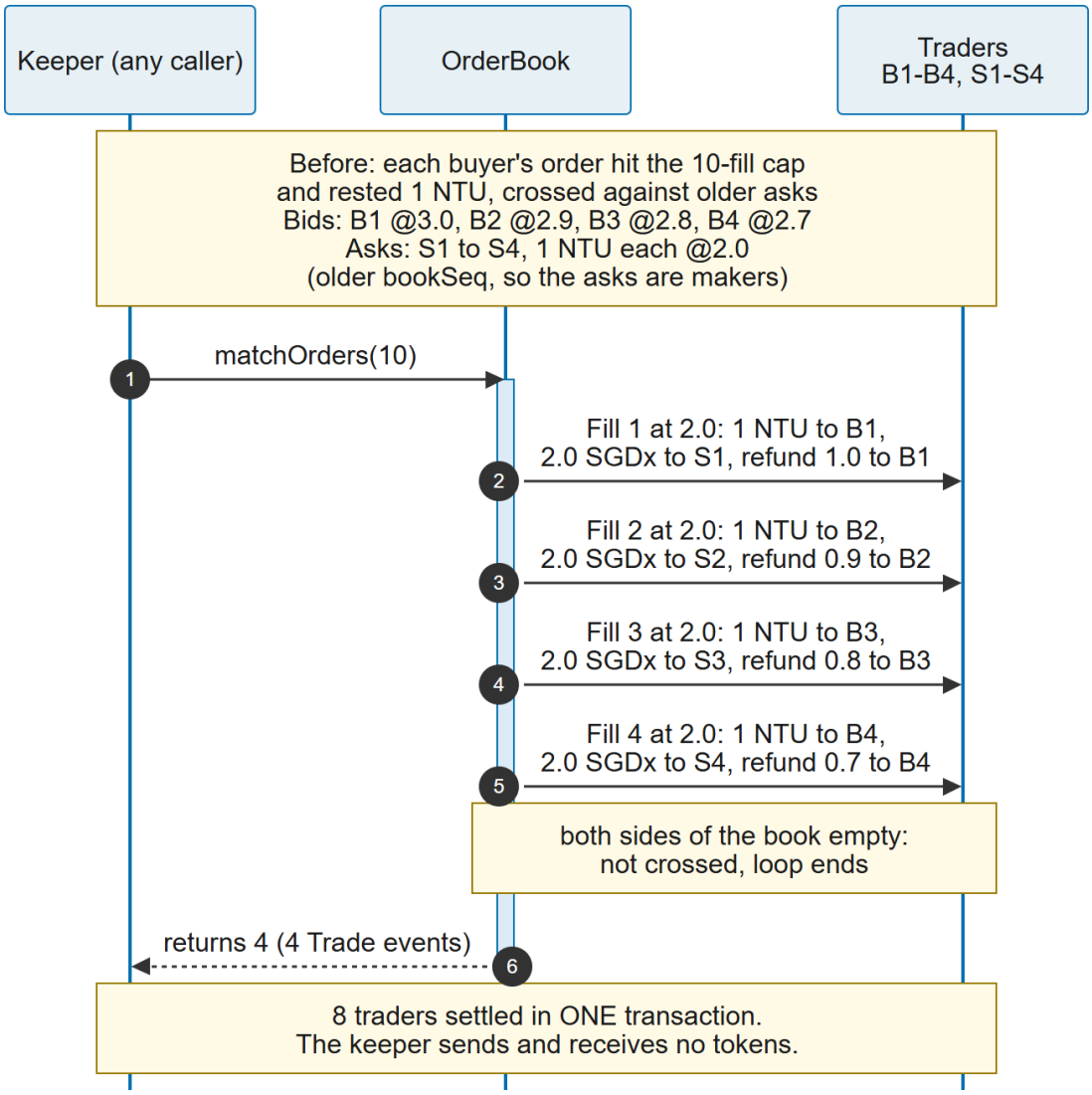


Figure 8: Multi-party batch match (Bonus 2). This replays `test_batch_multiPartySettlesFourBuyersAndFourSellersInOneCall`. One `matchOrders` call settles four buyers and four sellers at the makers' price and refunds each buyer's price improvement. The keeper's balances stay at zero.

4.3 What the AI proposed, and what happened to it

Area	AI proposal (the plan or Phase 0, 28 Oct, unless stated)	Outcome
Market structure	One `OrderBook` per pair; immutable base and quote; 18 decimals only	Accepted. Person 2's AI later proposed `MarketFactory` (Prompt 00, plan rev 3; built in P2-2), which the owner accepted as the listing mechanism (`prompt-log-person2.md`, Prompt 00 and P2-2)
Book	Sorted doubly linked lists, best order at the head, insert bounded by `MAX_INSERT_SCAN`	Accepted

Area	AI proposal (the plan or Phase 0, 28 Oct, unless stated)	Outcome
Priority	Price, then order ID	**Changed by the human (O1):** price, then an internal `bookSeq` stamped when an order enters the book, so a late-triggered stop queues fairly
Escrow order	SPEC (Phase 0): pull before matching. The Phase 2b prompt (1 Nov) said pull last (strict CEI)	**Human decision (1 Nov):** keep SPEC's order, pull before matching, which fills need. A second deviation, payouts during the match loop, was found by review (issue 002). Both are documented in SPEC section 5
Dust	`AmountTooSmall`	**Rejected:** `LOT_SIZE = TICK_SIZE = 1e9` makes every fill cost exact (A1)
Buy escrow after a fill	(gap c) A5: re-lock `requiredQuote(remaining, limit)` and refund the rest	Accepted
Settlement	Maker price, at most 10 fills per call, one `_settleFill`	Accepted; reused by batch and stops
Stops	Two sorted stop lists	Accepted; in Phase 6 they reuse the book's list code
Trigger budget	`maxCount × MAX_MATCHES_PER_TX`	**Changed by the human:** one shared budget per call
Activation into a too-deep book	Revert (`BookTooDeep`), Phase 6 implementation	**Changed after review (issue 006):** cancel and refund
Interface helpers	`requiredQuote` (Phase 0); `getOrder` returns zeros for an unknown ID (Phase 2b)	Accepted
Deployment and export	Script with an explicit deployer; an exporter tied to build artefacts and the live chain	Refined before and after review: the deployer design changed after a Context7 check of the Foundry docs; tying ABIs to deployed bytecode came from a Low-Medium review finding; P2-013 and P2-014 hardened the guard and the exporter
Front-end state	A pure lifecycle reducer (`web/lib/order-flow.ts`), shared by order, approval, token creation and listing, with runners that simulate, sign, wait and reconcile	The AI proposed the whole design in P2-5 (`prompt-log-person2.md`) and the owner approved the plan; the first version had gaps that review and testing later closed (P2-006, P2-007, P2-008 and P2-012)

---

5. Problems Discovered and Vulnerability Analysis

5.1 Inventory (most severe first)

ID	Area	Problem	Class	Severity	Status
006	Engine	One unplaceable stop blocked every stop activation, on both sides, for 102 wei	Gas bound becomes denial of service	High (availability)	Fixed (`1adcc59`)

ID	Area	Problem	Class	Severity	Status
004	Engine tests	Three injected matching bugs (overfill, phantom fills, wrong Trade fields) would have passed the suite; the code itself was correct	Test quality	High (tests)	Fixed (`6d1d14c`)
007	Engine	A dust front-run sandwiches a limit order through the fill cap, so the victim pays up to their limit	Economic / ordering	Medium	**Accepted by design**; pinned by a test
002	Engine spec	Second CEI deviation (payouts during matching) undocumented	Reentrancy reasoning	Medium (docs)	Fixed
P2-013	Deploy script	Chain-id guard bypassable with `--chain 31337`	Unsafe deployment	Medium	Fixed
P2-014	Exporter	A partial broadcast was exported as complete	Integrity	Medium	Fixed
P2-015	Front end	Stale balances shown as live after the RPC died	Misleading state	Medium	Fixed
P2-016	Front end	A crossed book made the spread display throw and crashed the whole Trade page	Unhandled state (negative value)	Medium	Fixed (P2-8B.3, found by the new browser suite)
P2-017	Front end	A stop cancelled before activation was labelled "Activated" in My Orders	Misleading history (status used to infer a past event)	Low	Fixed (P2-8C, found by the owner's MetaMask run; no funds affected)
P2-006, P2-007, P2-008, P2-012	Front end	A late signature lost; a result not bound to the requested order; a failure after signing invited a duplicate escrow; replacement never detectable	Transaction lifecycle	Medium (P2-008 also Low for wording)	Fixed
P2-001, P2-003, P2-005, P2-009	Front end and tooling	Token errors not decoded; review survived an account switch; scratch deploy overwrote the exporter record; five misleading UI states	Usability, correctness, workflow	Medium for usability / displayed correctness / workflow (no funds at risk)	Fixed
003	Engine	An accepted order could panic when filled (`qty × price` overflow)	Integer math	Low	Fixed
005	Engine	`address(0)` treated as the owner of unknown orders	Access-control edge	Low	Fixed (`dab79f8`)

ID	Area	Problem	Class	Severity	Status
001	Engine	`_unlink` could empty one side of the book	List integrity	Latent	Fixed (`dfe7318`)
P2-002, P2-004, P2-010, P2-011	Front end	Crossed-remainder warning missed; no network-switch guidance; creation-flow findings; ghost row	UI correctness	Low (P2-002: Low to Medium)	Fixed

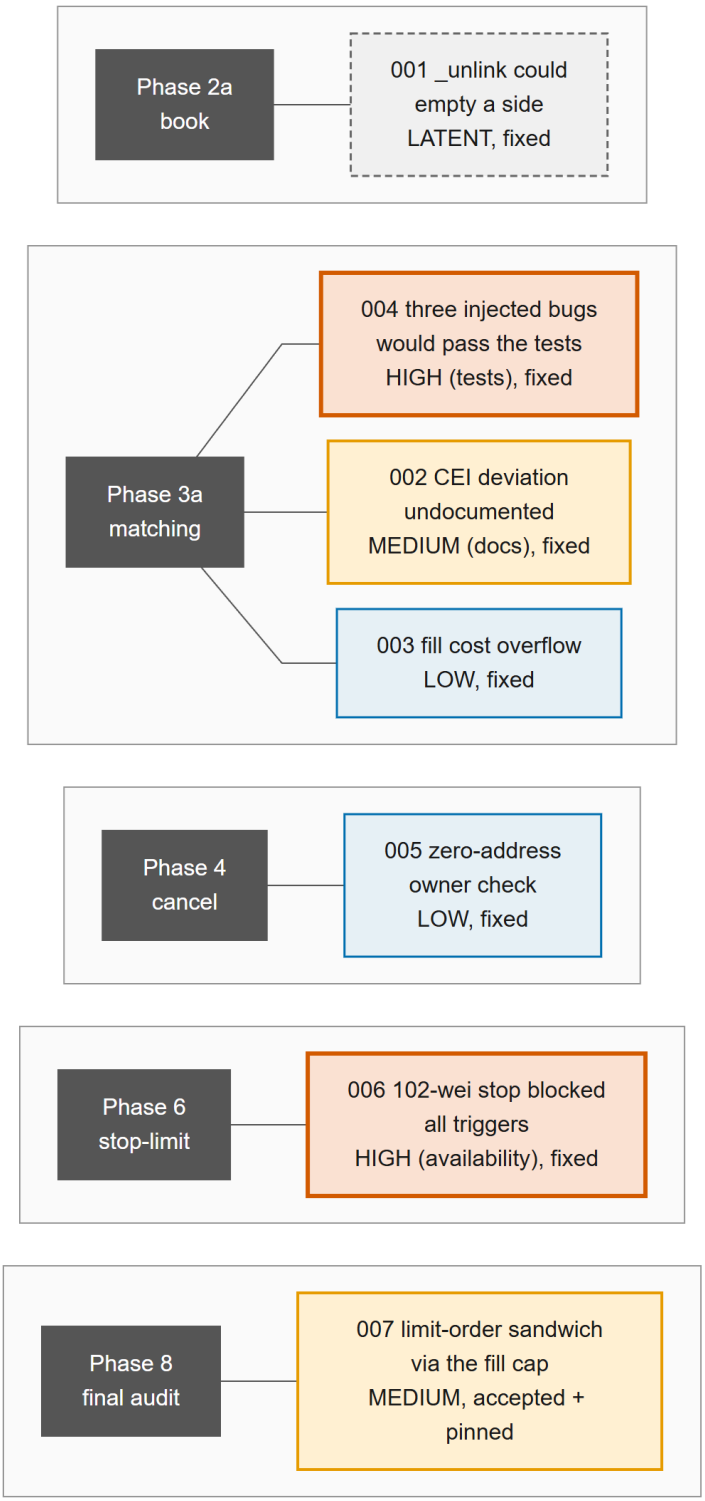


Figure 9: Engine issues by the phase that found them. The border shows severity (thick vermillion High, orange Medium, blue Low, dashed Latent), and every box also states it in words. Six issues are fixed, and issue 007 is accepted and pinned by a test.

5.2 Engine issues in the four-part format (Person 1)

The full text and the real before-and-after output are in each issue file.

006: one unplaceable stop blocked every trigger.

1. **Why the AI erred:** it reused the book's insert path, whose `BookTooDeep` revert is a gas bound, for stop activation. It missed that this reverts a permissionless call serving every stop, and that the failing stop stays at the head, so every later call fails too. It saw the revert but called it an accident.
2. **Impact:** a self-owned stop with the lowest trigger, plus 101 one-lot bids at 1 wei of escrow each (102 wei in all), made every `triggerStopOrders` call revert permanently. No stop-loss on either side could fire.
3. **Fix:** a non-reverting `_tryInsert`. An activated remainder that cannot enter the book is cancelled and refunded through the existing `OrderCancelled`. No ABI change. The human chose this option out of the three recorded.
4. **Evidence:** on the pre-fix line, 3 FAIL with `BookTooDeep()`; after the fix, 3 PASS. Mutant P6x (the pre-fix code) is caught by the unit suite and by the invariant suite alone.

004: matching test gaps.

1. **Why:** the tests were written from the prompt's list of shapes, never asking what would pass if the code were wrong.
2. **Impact:** three bugs would have passed green: overfilling a partly filled maker (paid from other traders' escrow), fills continuing after the taker was filled, and `Trade` reporting buyer and seller instead of maker and taker.
3. **Fix:** three targeted tests and a stronger `assertBookSorted`. In Phase 7 the issue's regression test was rewritten after review M3, because the first version could not fail on bug (a).
4. **Evidence:** 3 mutants SURVIVED (73 tests), then each fails one test. The Phase 7 regression fails on the pre-fix line (`ERC20InsufficientBalance`) and passes on the fixed code.

007: limit-order sandwich through the fill cap (accepted).

1. **Why:** two accepted limitations, the crossed book after the cap (O4) and dust using up fills, were analysed separately. Their composition against plain limit orders was written up only for stops.
2. **Impact:** a buy at limit 2.0 that expects an ask at 1.5 is front-run with 10 dust asks. It then pays 20e18 − 5.01e9 instead of 15e18, and the attacker makes 5e18 − 5.01e9. The loss is bounded by the victim's limit; solvency holds.
3. **Handling:** the human decided on 8 Nov not to change the engine at the end of the project. The attack is documented in SPEC section 8, with the mitigations not built (clear crossings first; do not rest a crossing remainder; a minimum fill), and the handoff advises the UI to

encourage tight limits.

4. **Evidence:** `test_issue007_dustSandwichMakesAHonestBuyPayItsLimit` reproduces it and pins the figures to the wei. They match the auditor's independent proof of concept.

002: CEI deviation understated.

1. **Why:** the rationale was written for one external call, before matching existed.
2. **Impact:** incomplete security reasoning. A callback during a payout sees a partly filled taker that is not yet in the book.
3. **Fix:** documented in SPEC section 5, pinned by view-spy tests, and both exceptions named in `CLAUDE.md`.
4. **Evidence:** a documentation fix, so by design there is no failing-then-passing pair. `test_payout_viewsShowFillSettledAndRemainderNotYetRested` pins the behaviour.

003: fill-cost overflow.

1. **Why:** the same formula was written two ways: escrow used `mulDiv` and settlement plain `q * p / 1e18`.
2. **Impact:** an order accepted at placement could never be filled. Only reachable with open-mint tokens.
3. **Fix:** settlement uses `Math.mulDiv`.
4. **Evidence:** `panic 0x11`, then PASS. The 512-bit differential fuzz of `requiredQuote` passed (Phase 7).

005: zero-address owner check.

1. **Why:** the AI assumed `trader != msg.sender` rejects unknown IDs, forgetting the zero address. `Open` is the enum's zero value.
2. **Impact:** a wrong error for unknown IDs, and `cancelOrder(0)` succeeding from `address(0)`. No funds at risk.
3. **Fix:** `order.trader == address(0) || order.trader != msg.sender`, reverting with `NotOrderOwner`.
4. **Evidence:** `OrderNotActive() != NotOrderOwner()`, then PASS. Mutant Z is caught in every smoke set.

001: `_unlink` could empty a side.

1. **Why:** the AI took `prev == 0` to mean "this order is the head", and left the precondition as a comment.

2. **Impact:** one missed check would hide every resting order on that side, with its escrow still held.
3. **Fix:** a link guard that reverts with the existing `OrderNotActive`, since the interface was frozen.
4. **Evidence:** 3 FAIL (one `0 != 1`, two missing reverts), then 3 PASS.

5.3 Person 2 issues in the four-part format

Each row summarises an issue file holding the full analysis and the real before-and-after output. Person 2's issues were found mostly by fresh-context reviews, real-chain runs and browser tests, and P2-017 by the owner's manual MetaMask run. None is an engine vulnerability; the severity of each is in the inventory above.

Issue	1. Why the AI erred	2. Why it is problematic	3. Fix	4. Evidence
P2-001 token errors not decoded	the test built the error object by hand; it assumed viem decodes any revert	no approval or a low balance showed a generic message	token ABI errors added to the simulation ABI	the regression test replays the real revert data (red, then green)
P2-002 fill cap hides a crossed remainder	read exactly 10 makers, not the cap plus one	the crossed-remainder warning failed exactly when needed	read the cap plus one	red then green; real chain with 12 makers
P2-003 review survived an account switch	the ticket kept its own `reviewing` state	the previous wallet's data shown as the new one's	review ends on an account, chain or market change	browser run failed, then 13 passed (mock wallet)
P2-004 no guidance on the ticket's network switch	a second hook instance with separate state	a hidden wallet popup was unexplained	the same waiting notice on the ticket	browser run failed, then passed
P2-005 scratch deploy overwrote the exporter record	missed that `--broadcast` overwrites an ignored record	a healthy chain reported as stale	record restored; scratch deploys redirect their output	exit 1, then passing
P2-006 "Try again" lost a late signature	an unconditional reset	two escrowed orders, the first invisible	abandoned attempts remembered	red reducer tests, then green
P2-007 result not bound to the order	any receipt with one `OrderPlaced` accepted	"Order confirmed" for a transaction the user did not sign	compare side, price and amount; replacements count as failures	red, then green

Issue	1. Why the AI erred	2. Why it is problematic	3. Fix	4. Evidence
P2-008 failure after signing invited a duplicate	every post-hash error treated as retryable	a second escrow after an RPC drop	a "may still be mined" warning; correct revert wording	red, then green
P2-009 five places the UI could mislead	happy-path reasoning	the UI stated things it did not know	five targeted fixes	no automated "before" (stated); after: type check, lint, build, browser assertion
P2-010 creation-flow review findings	invisible characters listed by code point, and others	look-alike symbols, a stale link	Unicode categories, hash tracking, failure state	red-first tests; real chain and browser
P2-011 ghost row on market switch	a panel keyed by symbol	a misleading extra row	key by position	`12 passed, 1 failed`, then `13 passed`
P2-012 replacement never detectable	a fake `waitForReceipt`; viem never given sender and nonce	a sped-up order reads "not confirmed", inviting a duplicate	pass sender and nonce; evidence-based chain-reset detection	red `10 failed`, then green; real chain `3 failed`, then `9 passed`
P2-013 chain-id guard bypass	reasoned from typical Forge behaviour, never tried hostile flags	broadcast to a non-local chain (11 transactions to a chain-1 node)	`run()` also asks the node with `eth_chainId`	the command now fails with `WrongChain(1)`; unit test; mutants caught
P2-014 exporter accepted a partial broadcast	"every check" left undefined	an interrupted deployment exported as complete	nothing pending, one receipt per transaction, all successful	unit tests; mutants E4 and E7 caught
P2-015 stale data shown as live	freshness designed around events; a dead node produces none	old balances shown as current before signing	a health probe, "last known" labels, transactions disabled while offline	controlled reproduction before and after; 17 + 7 unit tests; browser 11 passed
P2-016 a crossed book crashed the spread display	the spread was `ask - bid` on the assumption the book is never crossed; the fix for P2-002 updated only the ticket warning, not every other consumer of the book	the page that should offer Match Orders could not be drawn while the book was crossed	`bookSpread` returns none, crossed or an unsigned spread; the UI says "Crossed"	the first `e2e:match` run: 1 passed, 14 failed with `splitAmount expects an unsigned value`; then `crossed-book` unit tests and the suite green

Issue	1. Why the AI erred	2. Why it is problematic	3. Fix	4. Evidence
P2-017 a cancelled stop labelled "Activated"	a stop was modelled as pending or activated, from `getOrder` status alone; a pending stop that its owner cancels also leaves `PendingTrigger` without ever activating	a false statement about an on-chain history (order #3 never had a `StopOrderTriggered`)	`classifyStop` claims activation only from a `StopOrderTriggered` record, one bounded `eth_getLogs`, neutral wording when unreadable	before: the pre-fix build's text on #3 and the chain's logs; re-introducing the old rule turned 4 unit tests red; after: 15 unit tests, the real-chain test and the browser regression case (19 of 19) green

Person 2's P2-3 review also logged five lower findings without issue files (one Low-Medium, four Low, all fixed) and two Info notes. Two are detailed in `test-evidence/person2/p2-3-deployment.md`: ABIs not tied to the deployed bytecode, and `deployments.ts` not reproducible.

5.4 Reviews, static analysis and audits

- Engine Slither 0.11.6: 27 results, 0 real (13 false positives, 14 accepted). **`forge lint:`** 6 warnings, 0 real. See `phase8/slither-triage.md`.
- Person 2's Slither:
  - `DemoToken` and `TokenFactory`: 0 results
  - `MarketFactory`: 5 accepted naming notes
  - `Deploy.s.sol`: 6 accepted `calls-loop` notes (11 when re-run in P2-8A)
  - sources: `test-evidence/person2/p2-3-deployment.md` and `p2-8a-audit.md`
- Final independent engine audit (8 Nov):
  - no Critical or High
  - M-1 accepted as issue 007
  - three Low documentation fixes applied
  - Info notes passed to the handoff
  - See `ENGINE_SECURITY_SUMMARY.md`.
- Person 2's P2-8A audit and P2-8A.1: accessibility and RPC-cost findings from the audit; the liveness gap (R9, later P2-015) found by testing in P2-8A.1 (`p2-8a-audit.md`, `p2-8a1-fixes.md`).

5.5 Accepted limitations (not defects)

Listed in SPEC section 8:

- Stops: `lastTradePrice` is manipulable; stops can be sniped when the shared trigger budget runs out.

- Ordering and fill cap: a capped placement can leave the book crossed (O4); a limit order can be sandwiched through the fill cap (007); dust can grief the insert bound.
- Incentives: no reward for calling `matchOrders` or `triggerStopOrders`.
- Tokens and transfers: push payouts; 18-decimal tokens only; tokens sent directly to the contract are stranded.
- Trading rules: self-trades are allowed.

Person 2's front-end limits are in `docs/FRONTEND.md`, section "Known limits".

---

6. Validation Process and Testing Strategy

6.1 Layers

Layer	What it proves	Result (latest recorded)
Foundry unit, integration and regression tests	Behaviour against SPEC, with balances exact to the wei; factory market equal to a direct deployment	**335 passed** (245 engine, 90 Person 2) on `main` at `engine-final`
Engine reentrancy tests	Re-entry at every transfer reverts; swallow mode pays exactly once; view spies pin CEI	In the 245
Property fuzz	Random prices and sizes for place, cancel, batch and trigger, against reference arithmetic; `requiredQuote` differential checks (including 512-bit)	20 fuzz tests (15 engine, 5 Person 2): 1,000 runs at the default profile and 10,000 at deep. One exception: `testFuzz_coarsePlacementsKeepSolvencyAndBookOrder`, which is slow, runs 25 at default and 200 at deep
Stateful invariants	Solvency including pending stops; conservation; **an independent reference ledger equal to every balance**; order fields; lists; trade-price rules; uncrossed after `matchOrders`	All held: 256 × 64 (16,384 calls) and **1,000 × 128 (128,000 calls)**, 0 reverts
Mutation testing	The tests catch real bugs	Every non-equivalent engine mutant caught: each phase's set, then the full set of 67 on 8 Nov (65 caught, the 2 survivors R4 and P5q documented as equivalent; run). The invariant suite alone caught 15 of 17. Person 2's own checks (`test-evidence/person2/`): `MarketFactory` 6 of 6 (P2-2); the deploy script and exporter in P2-3, where every mutant was caught except D5, a documented equivalent, and three exporter mutants (E1 to E3) that survived until the review's missing tests were added; 5 of 5 in P2-6.1 and in P2-7; five deliberate breakages each in P2-8B.3 and P2-8B.4 turned a test red

Layer	What it proves	Result (latest recorded)
Coverage	Which code the tests execute	`OrderBook.sol` **100%** on lines (182/182), statements, branches (59/59) and functions
Static analysis	Known vulnerability patterns	Engine: 0 real (Slither and lint). Person 2's contracts and script: no real finding, only accepted naming and `calls-loop` notes (`p2-3-deployment.md`, `p2-8a-audit.md`)
Real-chain runs	Person 2's deploy script on a scratch Anvil; `cast` journeys for cancel, batch and stop-limit; `contracts/script/demo-journey.sh`	Every balance check passed
Front-end unit tests	Pure logic, reducers, runners with fake chains, decoding	604 passed, 76 gated skipped (P2-8C; 576 and 75 after P2-8B.4, 410 and 48 before the bonus UI)
Front-end real-chain and browser tests	Runners on a scratch chain; the UI in Chromium with a **mock** wallet	Real chain 76 passed (P2-8C; 75 in P2-8B.4); browser suites (trade, create, dashboard, hardening, liveness, cancel, match, stop, ux) 13 / 14 / 14 / 9 / 11 / 13 / 15 / 17 / 19 passed, 125 in all, all with a mock wallet (P2-8C; 106 in P2-8B.4)
Independent AI reviews	A fresh-context reviewer per engine phase from 3a and for Person 2's P2-1 to P2-6, plus a final audit of the engine only	Findings are in Section 5
Manual MetaMask	The real wallet	**Owner-run, owner-reported:** Tests 1 to 4 of `docs/TESTING.md`, a DTT token trade, and the happy path of each bonus: cancel (section 6.5), batch matching (6.7) and stop-limit orders (6.9). All three runs were re-read from the chain by the AI where the chain can show the facts. **Not performed:** the MetaMask rejection, account-switch-during-request, RPC-outage and chain-reset cases, the deep-book refund of a stop, and the remaining steps of `TESTING.md` (Section F lists each). We chose not to add MetaMask screenshots; the demo video shows the MetaMask flow



Figure 10: Verification evidence map for `OrderBook.sol` at `engine-final`. The evidence comes in four independent kinds: dynamic tests, static analysis, real-chain runs and AI review. Each figure is the latest recorded result, and the files are listed in the Evidence index.

6.2 Engine invariant suite

- Handler: 5 actors, a `duster` (bursts of one-lot makers, so cap hits and crossed books are common) and a `flooder` (101 one-lot bids outside the price cluster, which exercises the issue 006 path).
- Revert policy: only verified-legitimate reverts are tolerated. Anything else fails an invariant.
- Reference model: the handler replays every `Trade` and `OrderCancelled` with its own arithmetic into a per-address ledger, which the invariants compare with real balances. This replaced a first version whose ghost variables could not fail (Phase 7 review, H1).
- Reach (deterministic, 600 steps; `phase7/forge-test-default.txt`):

Path	Count
Crossing fills	840
Cap hits	13
Crossed-book calls	390
Cancels (open / partial / pending)	32 / 15 / 15
`matchOrders` calls that filled	32
Stop activations	69
Shared trigger budget used up	9
Auto-cancels	6
Violations	0

- Deep run (`phase7/forge-test-deep.txt`): all invariants held over 128,000 calls in 7,088 s. Calls per action were about 21,000 each.

6.3 Gas and size (engine)

- Maximum per call over the unit suites (`phase8/gas-report.txt`):
  - `placeLimitOrder` 1,808,075
  - `matchOrders` 781,693
  - `triggerStopOrders` 1,998,747
  - `cancelOrder` 215,166
- Every loop is bounded: 10 fills per call, 100 insert steps, 10 activations per trigger call.

6.4 What this does not prove

- Tests prove only what they check. Invariants were checked on random sequences up to depth 128, not exhaustively.

- Mutation results depend on the mutants written.
- Gas figures are measured cases, not upper bounds.
- Everything ran on a local Anvil, not a public network.
- Mock-wallet runs are not MetaMask. The manual MetaMask runs cover the happy path of each bonus only, and the owner's observations of what the wallet and the page displayed are not independently provable from the chain.
- No human security audit was done. All reviews were AI subagents.

6.5 Manual MetaMask evidence for Bonus 1 (2026-11-08)

The evidence below is grouped by source. **Owner-reported (real MetaMask, chain 31337):** Alice sold 5 NTU at 2; Bob bought 2 NTU at a limit of 2.2; it executed at the maker price of 2 (2 NTU, 4 SGDx, 0.4 SGDx returned); Bob's order Filled, Alice's PartiallyFilled with 3 NTU locked; Alice cancelled, the app showed the expected and event refund of 3 NTU, the transfer to Alice, escrow 0, and the order kept in My Orders; the final wallet held 969,998 NTU and 970,004 SGDx. **Cross-checked read-only by the AI against the owner's chain (8545):** order 1 Cancelled with filled 2e18, order 2 Filled, one `Trade` at 2e18 for 2e18 base and 4e18 quote, `OrderCancelled` refund 3e18, market escrow 0, both wallets' balances as stated (the 0.4 refund is inferred from the balances, not read from an event). **Not performed manually:** rejection, account switch during an in-flight request, cancelling a buy, RPC outage, chain reset. **Not in the repository:** the transaction hashes for this journey. Source: `person2/p2-8b2-cancellation.md` section 9.

6.6 Automated evidence for Bonus 2 (P2-8B.3, mock wallet and real chain; 2026-11-08)

Engine functionality (Person 1) is not re-claimed here. The new **front-end integration** was checked by: 50 new unit tests (crossed-book detection, the runner with a fake chain for success, no-op, revert, rejection, replacement, wallet cancellation, reset, timeout, RPC failure and account change, receipt decoding, error and wallet-prompt wording); a **real scratch-chain suite** (5 tests) reproducing the engine hand-off's scenario (Alice and Carol post 22 asks; Bob and the deployer each hit the 10-fill cap) in which a **keeper with no trading tokens** settles orders of four traders in one transaction, with the `Trade` events compared with an independent decode, exact token movements per trader, final statuses, escrow back to zero and the keeper's token balances unchanged; and a **mock-wallet browser suite** (15 cases: crossed and uncrossed books, review, wallet pending, result, repeated matching with nothing sent by itself, rejection, a book settled elsewhere, account switch, speed-up, wallet-cancelled request, outage with verified recovery, chain reset, mobile 375 px, console errors). axe-core found no violations in six states after one duplicate-id defect of the author's own was fixed. `forge test` was run once as a regression

check. **None of the automated evidence above is MetaMask:** the wallet is an injected stand-in. The real-wallet evidence is separate, in section 6.7. Failed first runs are kept in `person2/p2-8b3-batch-matching.md`; the one real defect they exposed in earlier AI-written code is P2-016.

6.7 Manual MetaMask evidence for Bonus 2 (2026-11-08)

The evidence below is grouped by source. **Owner-reported (real MetaMask, scratch chain 8546 through the scratch app on port 3100, keeper account `0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65`, which holds no NTU or SGDx):** Review match showed calldata `matchOrders(10)`, about 2 fills, a gas estimate of 277124 and no ERC20 approval; after one MetaMask confirmation the app reported block 78, 2 actual fills (order #45 with #47 and #46 with #48, each 1 NTU at 2 SGDx), 2 NTU and 4 SGDx settled, 1.5 SGDx returned to buyers, orders #45 to #48 Filled, the book no longer crossed, and 224189 gas used; `scripts/seed-crossed-book.sh verify-after` passed. **Independently verified by the AI with read-only calls afterwards:** transaction `0x52162ab7d8b916a6aba588cbc940e30a2cebfe2e620f597cf63c072a111ec779` in block 78, sent by the keeper (nonce 0) to the market with input `matchOrders(10)`, status success, gas used 224189, exactly two `Trade` events and six token transfers matching the app's report, orders #45 to #48 Filled, an empty book, the keeper's trading balances unchanged and its ETH down by exactly the gas; `verify-after` re-run, all checks passed. **Not performed:** the MetaMask rejection case, an account switch during an in-flight request, an outage and a chain reset in MetaMask. This confirms the happy path only. Source: `person2/p2-8b3-batch-matching.md` section 9. The earlier defect P2-016 (a crossed book crashed the page) and its fix are unchanged and remain in sections 5.1 and 5.3.

6.8 Automated evidence for Bonus 3 (P2-8B.4, mock wallet and real chain; 2026-11-08)

Engine functionality (Person 1) is not re-claimed here (`forge test` was re-run once: 335 passed). The new **front-end integration** was checked by: 79 new unit tests (the engine's eligibility rule including equality and the one-tick boundary, wording that keeps *waiting for price* and *eligible now* apart, validation and the limit-price escrow, the placement runner and the trigger runner with a fake chain for success, no-op, revert, rejection, replacement, wallet cancellation, reset, timeout, RPC failure, account change and the reconciliation problems, and the new receipt decoders); a **real scratch-chain suite** (11 tests, through the real runners) covering a pending sell and buy stop with exact escrow and no presence in the book, no trigger before a first trade, not-eligible stops, the exact boundary, an activation without and with a fill (the stop as taker at the bid's price, a token-less keeper sending it), cancelling a pending stop with the Bonus 1 runner, a bounded multi-stop call, the engine's **auto-cancel** of a stop that cannot enter a book with 101 orders ahead, and the 10-fill budget leaving a crossed book that the Bonus 2 runner clears, with a

solvency check after every scenario and the snapshot revert proven from the node; and a **mock-wallet browser suite** (17 cases). axe-core found no violations in 14 states, and the new text was read for contrast by sampling rendered pixels (all at or above 4.5:1; the pre-existing ticket button measured about 4.4:1, which P2-8C traced to its hover state, section 6.10). Five deliberate breakages each turned a test red. **None of the automated evidence is MetaMask:** the wallet is an injected stand-in, and at the time of this phase **no manual MetaMask test of Bonus 3 had been performed**; the owner ran the happy path later the same day (section 6.9). No new audit issue was found; the one failed browser assertion was a case-sensitive regex in the test itself. Source: `person2/p2-8b4-stop-limit.md`.

6.9 Real MetaMask evidence for Bonus 3 (owner-performed 2026-11-08; AI read-only cross-check)

The evidence below is grouped by source. **Owner-reported (real MetaMask, scratch chain 8547 through the scratch app on port 3101, chain id 31337, stop owner Anvil account 5, keeper Anvil account 6):** a buy stop-limit (#3, trigger 1.8, limit 1.7, 10 NTU, escrow 17 SGDx) was placed and then cancelled with the full 17 SGDx refund; a sell stop-limit (#4, trigger 1.8, limit 1.7, 10 NTU) was placed and shown as Pending Trigger and *Waiting for price* at a last trade of 2.0; the helper script moved the last trade to 1.9 (still waiting) and 1.8 (*Eligible now*, still pending and escrowed) and added a 4 NTU bid at 1.9; the keeper account reviewed `triggerStopOrders(10)` (estimate about 1 stop, gas estimate 227,434, no approval) and confirmed it; the app reported block 35, one stop activated, #4 partly filled for 4 NTU and 7.6 SGDx, a final last trade of 1.9 and 203,299 gas used; `scripts/seed-stop-orders.sh verify-after` passed every check. **Independently confirmed by the AI (read-only calls, no transaction sent, heads unchanged at 16, 78 and 35):** the full hashes, senders, nonces, calls and gas of every transaction (`TESTING.md` Test 14, section 14-4, and `p2-8b4-stop-limit.md`, section 9); `StopOrderPlaced` (and no `OrderPlaced`) for both stops; the cancellation's `OrderCancelled` and 17 SGDx transfer; the price path 2.0, 1.9, 1.8, 1.9 from `Trade` events; a simulated `triggerStopOrders(10)` returning 0 at blocks 24, 28 and 31, 1 at blocks 32 and 34 and 0 at block 35; one `StopOrderTriggered` in the chain's whole history (block 35, order #4, none for #3); one `Trade` at the **maker's price 1.9** (not the trigger 1.8 or the limit 1.7); #4 PartiallyFilled with the remaining 6 NTU resting at 1.7; the market holding exactly 6 NTU and 0 SGDx; the keeper holding no NTU or SGDx. **Not established independently:** what MetaMask and the app displayed, and which account the app was connected to at each moment. **Not performed:** MetaMask rejection, an account switch during a request, an RPC outage, a chain reset, the deep-book auto-cancel through the UI, another account cancelling a stop, `BookTooDeep`. **One defect found by this test (front-end correctness, low severity, no funds affected):** My Orders showed "Activated: it is now an ordinary order … (cancelled)" for the cancelled-before-activation stop

#3, which the chain contradicts. Why: `getOrder` cannot tell a stop cancelled while pending from one that was activated and then cancelled, and none of the AI's tests cancelled a pending stop and asserted that history line. This was fixed in P2-8C and recorded as issue P2-017 (section 6.10).

6.10 UX polish and the stop-history fix (P2-8C, mock wallet and real chain; 2026-11-08)

Engine functionality (Person 1) is not re-claimed here and no Solidity changed (`forge test` was not re-run). **What changed in the front end:** Title Case headings and column headers (one convention, enforced by a source-scanning unit test), accessible glass information tips (hover, keyboard focus, tap, Escape, outside click; copy pinned by a unit test against the engine's rules), semantic status badges with a separate "Eligible Now" chip, the Trigger Price and Limit Price of every stop shown in My Orders for the whole life of the order, a connected-account chip and a "Sending From" row in every review, and the stop-history fix of issue P2-017 (activation is claimed only from a `StopOrderTriggered` record, from one bounded, topic-filtered `eth_getLogs`, with neutral wording when it cannot be read). **Checked by:** 28 new unit tests (604 passed, 76 gated skipped), a real-chain test of the history reader (cancelled before activation, activated and filled, activated then cancelled by the owner, the engine's auto-cancel, one request, an RPC failure; 76 real-chain tests passed), a new 19-case browser suite on a fresh scratch chain (125 browser cases in all, mock wallet), axe-core with 0 violations on the changed states at 1586 px and 375 px, contrast measured from rendered pixels (all badges 5.6:1 or better), and no horizontal overflow at 375, 768 and 1280 px. **Limits:** none of this was seen in real MetaMask; the first version of the request-economy test counted another open page's requests and failed (a test mistake, fixed); the shared primary button's earlier 4.4:1 reading was reproduced only while hovered (4.51:1) and the hover lift was reduced (4.80:1); the engine's auto-cancel is covered by the real-chain suite, not by the browser suite. Details: `docs/ai-audit/test-evidence/person2/p2-8c-ux-polish.md`.

7. Improvements and Refinements

Engine (Person 1):

- All three bonuses, within the frozen interface:
  - `cancelOrder`: owner-first; exact refund of the remainder; also cancels pending stops.
  - `matchOrders`: anyone may call it and the caller gains nothing; multi-party settlement in one transaction.
  - Stop-limit: escrow at placement; sorted stop lists; one shared fill budget; a fresh `bookSeq` at activation; auto-cancel when a stop cannot be placed.

- One settlement path (`_settleFill`) and **one list implementation,** reused across features. Shared helpers instead of duplicated code (`_validate`, `_record`, `_pullEscrow`, `_closeAndRefund`).
- A guard the AI first left as a comment, now enforced in code at our insistence (001); a missed zero-address case closed (005). `mulDiv` everywhere (003).
- SPEC section 6 answers tricky point 1:
  - The taker matches inside `placeLimitOrder`.
  - Anyone can call `matchOrders` and `triggerStopOrders`.
  - Work per call is bounded, and there is no keeper reward.
- SPEC section 2 (`bookSeq`) answers tricky point 2.
- Verification beyond the first output:
  - a mutation script
  - an invariant suite with an independent reference ledger
  - a regression test per issue, each shown failing on the original bug where a code fix exists
  - real-chain verification
  - a one-command demo replay

Front end and tooling (Person 2):

- Safer order flow: exact approvals; simulation before signing; results bound to the requested order; duplicate guards.
- Accurate crossed-remainder warnings (P2-002).
- Transaction-replacement tracking and evidence-based chain-reset detection (P2-7).
- Unicode and decimals checks before signing.
- Deployment safety: the chain-id guard; an exporter that refuses incomplete, stale or mismatched data.
- `MarketFactory` as the listing mechanism.
- Multi-market dashboard.
- Accessibility, RPC-cost and liveness fixes (P2-8A.1 and .2).
- UI follow-up from the MetaMask run, done in P2-8C (section 6.10): a stop's history is read from its `StopOrderTriggered` and `OrderCancelled` records instead of being inferred from its status, My Orders shows the Trigger Price for the whole life of a stop, statuses are coloured badges, headings follow one Title Case convention, and small information tips explain the trading terms. No contract or ABI change.

- Cancelling in the front end (P2-8B.2): a Cancel action on open, partly filled and pending-stop orders that states the refund before the wallet is asked, sends one `cancelOrder` transaction, and reports success only after the confirmed receipt has been read back (the `OrderCancelled` event, the token transfer and the order's final status must agree). Cancelled orders stay in My Orders as history.
- Stop-limit orders in the front end (P2-8B.4): a Stop-Limit mode in the ticket (trigger and limit price, escrow at the limit price, exact approval, review), pending stops that are shown apart from the order book with *waiting for price* told apart from *eligible now: needs a Trigger stops transaction*, cancelling through the existing flow, and a permissionless Trigger Stops panel whose result comes only from the confirmed receipt and never calls activation a fill (it separates resting, partly filled, filled and the engine's own refund). It leaves the engine's accepted limitations visible: the trigger is the market's own last trade price, which trading can move, and a stop is not a guarantee of execution or price.
- Permissionless batch matching in the front end (P2-8B.3): a Match Orders panel on Trade, shown only while the market itself reports a crossed book, with review and simulation before the wallet, one deliberate `matchOrders(10)` request, and a result built only from the confirmed receipt's `Trade` events and cross-checked against `getOrder` and the token transfers. It lets a token-less keeper settle orders of several traders in one transaction (shown on a real scratch chain with four traders), and it never repeats by itself. The engine's accepted economic limitation on capped placements (issue 007) is unchanged; the UI does not claim to remove it.
- Local-deployment troubleshooting (follow-up to P2-8B.2): a MetaMask nonce gap after an Anvil restart (the wallet kept the former chain's nonces, so transactions stayed pending) was diagnosed in the development environment. It is a wallet and node synchronisation issue, not a contract vulnerability, and is separate from the earlier leftover-allowance test contamination. The response was documentation only: `README.md` and `docs/DEPLOYMENT.md` now separate starting Anvil, `forge build`, deploying, exporting, `scripts/check-local.sh` and starting the frontend, and give read-only `cast` diagnostics and a recovery order (inspect first; drop an identified pending transaction and clear wallet activity only on a disposable local chain).

---

8. Lessons Learned

Sections 8.1 to 8.4 draw mainly on the engine work (Person 1) and give the recommendations for future AI-driven dApp development; the "Person 2 additions" at the end of 8.4 and Section 8.5 come from the front-end and tooling work (Person 2).

8.1 Where AI excelled

Volume and discipline of testing. It would not have been realistic to write all of this by hand in the time available (Figure 10):

- 245 engine tests (335 Foundry tests across the project)
- 100% line and branch coverage of `OrderBook.sol`
- 15 engine fuzz tests
- an invariant suite that held over 128,000 random calls

Independent review found real bugs. From Phase 3a, we had a separate reviewing agent check each phase. It ran with no context from the implementing session. This was the most valuable practice we used: it found faults that the implementing agent's own tests had missed (Figure 9). Two examples:

- Issue 004: the tests had real gaps. Three injected bugs, including one where a partly filled order is overfilled and paid out of other traders' escrow, would have passed all 73 existing tests. The code itself was correct; the reviewer found the gaps by mutation testing.
- Issue 006: a High-severity design flaw. About 102 wei of dust could permanently block every stop-limit trigger. The reviewer proved it by running the attack in a scratch copy of the contracts.

Testing the tests. Mutation testing deliberately breaks the engine and checks that the tests fail. It showed where our tests were weak. For example, removing the status check in cancellation (mutant C1, Phase 4) went unnoticed until we added a test that catches it.

Verification on a real chain. The agent deployed the system on a scratch Anvil chain with Person 2's deploy script. It then replayed the brief's journey (placement, partial fill, cancel and a multi-party batch match) with real transactions, and later added cast journeys for the stop-limit orders. This confirmed that the factory, engine and scripts work together, not just in unit tests.

8.2 Where AI failed or needed correction

Design decisions that looked fine but were not.

- Small orders: the agent first proposed rejecting orders whose total cost rounded to zero. That missed dust created by individual partial fills. We replaced it with lot and tick sizes, which make every fill cost exact. The rounding rules stay only as a safety net.
- Stop-trigger budget: it first proposed a fill budget per activated order (`maxCount × MAX_MATCHES_PER_TX`), which still allowed a long, expensive call. We changed it to one shared budget.

Understating severity. In the spec, the agent described the stop-trigger denial of service (issue 006) as an occasional deep-book accident. Only when the reviewer ran it did it become clear that this was a cheap, permanent attack.

Inaccurate reporting. The agent sometimes reported results incorrectly:

- An understated "worst case" gas figure: 1.28M. The reviewer's worst-case test measured 1.78M at the time; on the final code the same test measures 1.81M.
- A deep invariant run reported as dead while it was still running. It later completed and passed, and the agent corrected its report before merging.
- A commit message (`88cedf7`) that overstated how many fuzz runs one test used. The agent flagged this itself in Phase 8.

We learned to ask for raw tool output, not summaries, before trusting a claim.

Tests that did not test what they claimed. These tests passed, but proved less than their names suggested:

- Unreached cases: the original fuzz test never reached the fill cap or a crossed book (0 of 500 seeds, by the reviewer's replay). The agent's first replacement reached neither in 40 seeds until it was redesigned.
- Temporary behaviour pinned: one test pinned behaviour from an earlier phase that the prompt had called temporary.
- Solidity mistakes: one test aliased a memory struct.

Spec and code drifting apart.

- Escrow order: the agent once implemented an escrow order that contradicted `SPEC.md` because the prompt said otherwise (Phase 2b). It flagged the conflict itself, citing SPEC, which `CLAUDE.md` names as the source of truth, and we decided to keep SPEC's order.
- CEI exceptions: a spec section claimed one exception to checks-effects-interactions when there were two. The independent reviewer caught this one (issue 002).

8.3 What we would keep

1. **Write the specification and freeze the interface before writing the logic.** It let two people work in parallel and kept the agent from inventing functions.
2. **Plan mode first, then small phases.** Each phase was small enough to review, ended with green tests, and was merged separately. That kept the history readable for this report (Figure 2).

3. **An independent reviewer for every significant change,** with no shared context, so the work is never graded by the agent that wrote it (Figure 1).
4. **Mutation testing,** to measure test quality, not just test count.
5. **Humans make design trade-offs.** The independent reviewers and the final auditor surfaced the forks: keeping a crossed book, documenting dust griefing, auto-cancelling blocked stops, accepting issue 007. The decision and the responsibility stayed with us.
6. **Log as you go.** Because the prompt log and the issue files were written during development, this report was a matter of assembly, not reconstruction.
7. **Separate planning and review from implementation.** Using the claude.ai chat for planning and review, apart from the coding sessions, gave a second opinion at every step. Neither tool was treated as a source of truth: every claim was checked against the repository.

8.4 Recommendations for future AI-driven dApp development

- Treat every AI claim as unverified until you have seen the command output that supports it.
- Ask the AI to attack its own design, not only to test it. Our most serious finding, the issue 006 denial of service, came from a reviewer, not from the implementing agent's own tests.
- Budget time for long verification runs. The deep invariant run took about two hours (7,088 s), and heavy runs such as the full mutation set can exceed tool timeouts.
- Agree interfaces between teammates in writing, early. Our verbal agreement held, but it left no record for the audit trail.
- Expect platform friction on Windows (line endings, PATH, long file names), and fix it once, in project configuration.
- Person 2 additions (from the front-end work, Section 8.5):
    - Plan time for integration and testing, not only for generation. Run small, targeted tests while implementing, and keep the full regression for milestones.
    - Isolate destructive or long-running tools (mutation, deep runs, scratch chains) and verify the source against `HEAD` afterwards.
    - Keep the kinds of evidence apart: automated tests, owner-run manual sessions (cross-checked read-only against the chain where it can show them) and what has not been done.
    - Read the other chains before writing a manual procedure: every local Anvil shares chain id 31337 and MetaMask caches nonces per chain id.

8.5 Person 2's lessons

These lessons come from the front-end, deployment and tooling work (issues P2-001 to P2-017, `prompt-log-person2.md` and the evidence files in `test-evidence/person2/`). Engine findings belong to Section 5.2 and are not repeated here.

Generating code is not finishing the feature. The AI produced working front-end code quickly, but a feature was only done when it had been tested, debugged, integrated with the others and documented, and that part often took longer than writing the code. Each bonus interface needed new unit tests, a real-chain suite, a browser suite, a manual MetaMask procedure and documentation before it counted as built. The P2-8C polish phase is the clearest case. It changed headings, labels and badges, which sounds small, yet it took the owner more than an hour because the changed text was quoted in assertions across eight existing browser suites, and the whole regression (604 unit tests, 76 real-chain tests, 125 browser tests) had to be run again to show that nothing else had moved.

Tests tied to wording need deliberate care. Several browser assertions failed after the capitalisation change only because they looked fields up by their text. One lookup by the label "Limit price" also matched the new information button named "About Limit Price", so it was replaced by the field's stable identifier. The lesson is to separate two kinds of test. Behaviour tests should use stable identifiers so that a wording change does not break them. A small number of tests should check the visible wording and accessible names on purpose: in P2-8C a unit test scans the source for the Title Case convention and a browser test reads every page's headings from the rendered DOM.

Contract correctness does not cover the interface. The engine legitimately allows a crossed book after a capped placement (SPEC section 8). The dashboard still computed the spread as `ask - bid` on the assumption that the book is never crossed, so a crossed book crashed the whole Trade page (P2-016). The earlier fix for the same state (P2-002) had updated one warning and not every consumer of the book. A new feature creates states the older code never saw, and each consumer of the data has to be checked against them.

A real wallet shows what a mock cannot. The mock-wallet suites passed while two problems were still there, and both appeared only when the owner used MetaMask:

- After an Anvil restart MetaMask kept the former chain's nonces, so approvals stayed pending. It was an environment and wallet-synchronisation problem, not a code defect, and it is documented in `docs/DEPLOYMENT.md` and `docs/TESTING.md` with read-only diagnostics first and a reset only on a disposable chain.

- Cancelling a pending stop-limit order exposed the label "Activated" for an order that had never activated (P2-017). No automated case had cancelled a pending stop and then checked its history line, and the chain's logs contradicted the label.

Several chains with one chain ID. The Anvil instances on ports 8545, 8546 and 8547 all used chain ID 31337 but kept separate histories. MetaMask identifies a network by chain ID, so which chain the wallet was talking to depended only on its RPC URL, and the same address could have different balances, allowances and nonces on each chain. The manual procedures therefore name the exact RPC and frontend port of each environment, and the Bonus 3 test used accounts that had never sent a transaction on any of the chains so that no cached nonce could be stale.

The account in MetaMask is not necessarily the account on the site. Selecting another account in the extension does not connect that account to the website. During the Bonus 3 run the owner took the page still showing the previous account's orders for an isolation fault. Checking the actual wallet connection resolved it, and no defect was recorded. P2-8C made the connected address visible in My Orders and in every review, with a short explanation, and the existing safeguards (queries keyed by account, in-flight actions ended when the account changes, P2-003) were left as they were.

The working environment is part of the work. Developing under WSL meant learning the differences between PowerShell and the Linux shell, which directory a command runs in, where Node and Foundry are installed, how to find and stop a process, and which port each server uses. Small examples are in the record: the first `forge test` needed `--root .` (`prompt-log-person2.md`, Prompt 00), Playwright's Chromium needed shared libraries that WSL lacked (`docs/TESTING.md`, section E), and port conflicts between the app and the scratch servers needed their own troubleshooting entry. These costs are easy to underestimate when planning time.

Review, mutation and failing-first tests removed wrong assumptions. Fresh-context reviews of the deployment work found P2-013 and P2-014, which the author's own tests had not, and the review of the trading code returned 13 findings. Mutation checks showed where tests were too weak: three exporter mutants survived until the missing tests were added, and a first mutation script gave a false "survived" because it changed a comment instead of the code. Regression tests written to fail first, for example the replacement-detection and result-binding tests (P2-007, P2-012) and the order #3 case of P2-017, kept the same mistakes from returning. Tests built on hand-made fakes proved only the fake (P2-001, P2-012), and reasoning from the typical case missed the hostile one (P2-013).

Different evidence proves different things. The report keeps four kinds apart. Mock-wallet browser tests check the application's logic. Real-chain runs execute the application's transaction code against real contracts. The owner's MetaMask sessions show what a person sees with a real wallet. The AI's read-only cross-checks of the chain confirm hashes, events and balances, and

cannot confirm what MetaMask or the page displayed. Treating them as interchangeable would have overstated what was verified: for example, an earlier note that the primary button had a contrast of 4.4:1 was not reproducible at rest and applied to its hover state. The sessions that were run are recorded as run, and the cases that were not (the MetaMask negative cases) are recorded as not run.

Documentation needs review as well. Drafts carried stale feature statuses after the implementation had moved on: a requirements-traceability row still said "MetaMask not tested" after the owner's sessions, and an earlier README line described invariant tests that did not exist. Documentation therefore needs the same checking against the repository as code.

Branches and shared documents drift apart. Person 1 and Person 2 worked on separate workstreams and branches. When `main` was merged into `faheem` on 8 Nov, `main` had twelve commits that `faheem` lacked, and both branches had edited this report and the video plan. The merge produced two conflicts in this report, in the executive summary and in Section 8, which had to be resolved by hand without losing either teammate's verified text. Synchronising earlier and more often would have kept those conflicts small.

What Person 2 would do differently. Allocate explicit time for integration and testing in each phase instead of treating it as the tail of implementation. Run small, targeted tests while implementing and keep the full regression for milestones; running the unit tests with the real-chain variables set, which ran the gated suites in parallel and dirtied a scratch chain in P2-8C, shows the cost of a broad run at the wrong moment. Use stable identifiers for behaviour tests. Reconcile documentation with the code at every phase, not only at the end.

Requirements traceability

Req.	Brief requirement	Code	Tests	Evidence	Status
M1	Asset issuer: issue new ERC20 tokens	`DemoToken.sol`, `TokenFactory.createToken`	`DemoToken.t.sol`, `TokenFactory.t.sol`	`person2/p2-1-issuance.md`	Done
M2	Token listing	`MarketFactory.createMarket`; List market page	`MarketFactory.t.sol`, `MarketFactoryIntegration.t.sol`, `Integration.Factory.t.sol`	`person2/p2-2-listing.md`; `phase8` (engine side)	Done

Req.	Brief requirement	Code	Tests	Evidence	Status
M3	Submit buy and sell limit orders	`OrderBook.placeLimitOrder`; `web/components/trade/`	`OrderBook.Place.t.sol`, front-end order tests	`phase7/forge-test-default.txt`; `person2/p2-5-trading.md`	Done
M4	Automatic execution of matched orders	`_matchTaker`, `_settleFill`	`OrderBook.Match.t.sol`, invariant suite	`phase7`	Done
M5	Partial fulfilment	same; A5 refunds	`test_match_*PartlyFilled*`, fuzz	`phase7`; demo step 1	Done
M6	Self-custody, non-custodial	no admin; escrow leaves only to the counterparty or the owner	invariants (ledger, conservation); Slither	`ENGINE_SECURITY_SUMMARY.md`	Done
M7	Simple front end	`web/`	Unit, real-chain and browser suites: 604 unit (76 gated skipped), 76 real-chain and 125 mock-wallet browser tests (section 6.1; 410 and 48 before the bonus UI)	`person2/p2-4` to `p2-8c`	Done
B1	Bonus 1: cancel unfilled amounts	`cancelOrder`, `_closeAndRefund`	`OrderBook.Cancel.t.sol` (52), regression 005	issue 005; scratch-chain cast; demo step 2	**Engine done. UI: built and verified with a mock wallet on a scratch chain in P2-8B.2** (Person 2; evidence); **MetaMask:** the happy path was run by the owner on 2026-11-08 (section 6.5); other manual cases pending

Req.	Brief requirement	Code	Tests	Evidence	Status
B2	Bonus 2: batched multi-party matching	`matchOrders`; UI: `web/lib/crossed-book.ts`, `match-runner.ts`, `receipt.ts` (`decodeMatchReceipt`), `web/components/trade/match-panel.tsx`	`OrderBook.Batch.t.sol` (31), `test_batch_multiParty*` (4 + 4 traders, one transaction); UI: `crossed-book.test.ts`, `match-runner.test.ts`, `match.integration.test.ts` (5, real chain), `e2e/match.e2e.mjs` (mock wallet)	demo step 3; handoff; evidence	**Engine done. UI: built and verified with a mock wallet and on a real scratch chain in P2-8B.3** (Person 2); **MetaMask:** happy path confirmed by the owner on a scratch chain (2026-11-08; section 6.7); rejection and account switch not run (`docs/TESTING.md` Test 13)
B3	Bonus 3: conditional orders	`placeStopLimitOrder`, `triggerStopOrders`; UI: `web/lib/stop-orders.ts`, `stop-runner.ts`, `trigger-runner.ts`, `receipt.ts` (`decodeStopReceipt`, `decodeTriggerReceipt`), `web/components/trade/order-ticket.tsx`, `stop-result.tsx`, `trigger-panel.tsx`, `my-orders.tsx`	`OrderBook.Stop.t.sol` (42), regression 006; UI: `stop-orders.test.ts`, `stop-runner.test.ts`, `trigger-runner.test.ts`, `stop.integration.test.ts` (12, real chain), `stop-history.test.ts` (15), `e2e/stop.e2e.mjs` (17) and `e2e/ux.e2e.mjs` (19, mock wallet)	issue 006, P2-017; demo step 4; evidence, P2-8C evidence	**Engine done. UI: built and verified with a mock wallet and on a real scratch chain (P2-8B.4, polished in P2-8C)** (Person 2); **MetaMask: happy path run by the owner on a scratch chain on 2026-11-08 and chain-verified read-only (`docs/TESTING.md` Test 14, section 6.9); rejection, account-switch, outage, chain-reset and deep-book-refund cases not run.** The cancelled-stop label defect that run found was fixed in P2-8C (P2-017)
T1	Tricky point: who triggers execution, and how often	SPEC section 6	`test_batch_thirdPartyCallerGainsAndLosesNothing`, trigger tests	SPEC section 6	Answered
T2	Tricky point: priority of older orders	`bookSeq`, `_ranksAhead`	FIFO tests; `test_batch_makerFollowsBookSeqNotOrderId`; `test_trigger_activatedStopQueuesBehindEarlierOrdersAtItsPrice`	SPEC sections 2 and 7 (O1)	Answered
D1	AI Audit Report, eight sections	this file	n/a	n/a	Done

Req.	Brief requirement	Code	Tests	Evidence	Status
D2	Four-part breakdown per major issue	`docs/ai-audit/issues/` (001 to 007, P2-001 to P2-017)	n/a	Section 5	Done
D3	Validation evidence	n/a	n/a	Section 6; evidence index	Done
D4	Manual tests	`docs/TESTING.md`	n/a	Section F	**Partly done:** the owner ran Tests 1 to 4, a DTT trade and the happy path of each bonus; the negative cases were not run. We chose not to add MetaMask screenshots; the demo video shows the MetaMask flow
D5	Peer evaluation (mandatory, confidential: brief sections 4 and 7)	not in the repository	n/a	n/a	**Each student submits their own**

Evidence index

Claim area	File
Engine prompt history, decisions, the AI's mistakes	`docs/ai-audit/prompt-log.md`
Person 2's prompt history	`docs/ai-audit/prompt-log-person2.md`
Tools and models per phase (Person 2)	`docs/ai-audit/tooling-and-methodology.md`
All issues in four parts	`docs/ai-audit/issues/`
Behaviour specification and limitations	`SPEC.md`
Frozen interface	`CONTRACT_INTERFACE.md`
Engine phases and prompts	`docs/ENGINE_PHASES.md`
Engine security case	`docs/ai-audit/ENGINE_SECURITY_SUMMARY.md`
Full test run (default profile)	`test-evidence/phase7/forge-test-default.txt`
Deep invariant run	`test-evidence/phase7/forge-test-deep.txt`
Mutation results	`test-evidence/MUTANTS.md`, `phase7/mutants-phase7.txt`
Slither and lint	`test-evidence/phase8/slither.txt`, `slither-triage.md`

Claim area	File
Coverage, gas, sizes	`phase8/coverage-summary.txt`, `gas-report.txt`, `sizes.txt`
Report verification (numbers re-derived, links, hashes, issue IDs, markers)	`test-evidence/wrapup/report-verification.md`
Debugging and replay	`test-evidence/DEBUGGING.md`
Frontend handoff (signatures, flows, stale files)	`docs/ENGINE_HANDOFF.md`
Demo replay script	`contracts/script/demo-journey.sh`
Person 2's audit and evidence	`test-evidence/person2/` : P2-1 to P2-7, the P2-8A audit and fixes (`p2-8a-audit.md`, `p2-8a1-fixes.md`, `p2-8a2-liveness.md`), the bonus phases (`p2-8b2-cancellation.md`, `p2-8b3-batch-matching.md`, `p2-8b4-stop-limit.md`) and P2-8C (`p2-8c-ux-polish.md`, `p2-8c-screens/`)
Original engine plan (pre-implementation scope)	`docs/DEX_Trading_Engine_Plan.md`
Beginner's guide to running and testing the exchange	`docs/BEGINNERS_GUIDE.md`
Manual testing guide	`docs/TESTING.md`